

Research paper

Guided deep reinforcement learning framework using automated curriculum scheme for accurate motion planning

Deun-Sol Cho ^{a,1}, Jae-Min Cho ^{a,1}, Won-Tae Kim ^{b,*}^a Major of Future Convergence Engineering, Koreatech University, Cheonan-si 31253, Republic of Korea^b Computer Science and Engineering, Koreatech University, Cheonan-si 31253, Republic of Korea

ARTICLE INFO

Keywords:

Curriculum learning
Deep reinforcement learning
Motion planning
Robotic arm
Unsupervised learning

ABSTRACT

Collaborative robotic arms in smart factories should ensure the safety and interactivity during their operation such as reaching and grasping objects. Especially, the advanced motion planner including the path planning and the motion control functions is essential for human-machine co-working. Since the traditional physics-based motion planning approaches require extreme computational resources to obtain near-optimal solutions, deep reinforcement learning algorithms have been actively adopted and have effectively solved the limitation. They, however, have the easy task preference problem, primarily taking the simpler ways for the more rewards, due to randomly training the agents how to reach the target points in the large-scale search spaces. Therefore, we propose a novel curriculum-based deep reinforcement learning framework that makes the agents learn the motion planning tasks in unbiased ways from the ones with the low complexities to the others with the high complexities. It uses the unsupervised learning algorithms to cluster the target points with the similar task complexities for generating the effective curriculum. In addition, the review and buffer flushing mechanisms are integrated into the framework to mitigate the catastrophic forgetting problem where the agent abruptly lose the previous learned knowledge upon learning new one in the curriculum. The evaluation results of the proposed framework show that the curriculum significantly enhances the success rate on the task with the highest complexity from 12% to 56% and the mechanisms improve the success rate on the tasks with the easier complexities from an average of 66% to 76.5%, despite requiring less training time.

1. Introduction

A high degree of freedom (DoF) robotic arm with multi-functional capabilities is recognized as a key component for the autonomy of the future industries. The robotic arm is a physical system that performs rotational and linear motions within its workspace, based on a series of interconnected links and joints (Ekrem and Aksoy, 2023; Barjuei et al., 2024). Each link can have varying properties according to the length, shape, and weight, and it can be joined to other links through joints. Each joint facilitates the rotation of the connected links, enabling the end effector of the robotic arm to reach a desired target position.

To control the physical dynamics of the robotic arm, a motion planning method is necessary. This method includes path planning and the motion control modules (Liu and Liu, 2021; Behere, 2010). The former module plans the moving path of the robotic arm from the start position to the goal position by considering factors such as kinematic characteristics (i.e., the length of the links, the constraints of the joint angles) and obstacles. The latter module adaptively controls the joints

of the robotic arm through feedback signals so that it can reach the target position along the defined path.

These modules can face the challenges related to the computing resources, due to the robotic arm's complex movements and the non-linear nature of the physics-based governing equations (Liu et al., 2023a, 2021). This challenge becomes significantly greater if the robot has a complex geometry or if the target point for the motion planning is in a hard-to-reach area. This is a major reason for their limited utilization in smart factories, where large number of the robotic arms with complex geometries are widely used and their control objectives are flexibly changed in a timely manner.

To address this challenge, researchers have conducted studies on applying the artificial intelligence (AI)-driven approaches to the robotic arm motion planning (Annaswamy, 2023; Tang and Hauser, 2017; Guan, 2020; Yang and Wang, 2022a; Zhou et al., 2021b,a; Li et al., 2020; Yang et al., 2021; Man et al., 2021; Ni et al., 2021). The existing AI-driven approaches have been effective in training the agents to

* Corresponding author.

E-mail address: wtkim@koreatech.ac.kr (W.-T. Kim).¹ Deun-Sol Cho and Jae-Min Cho are co-first authors.

Table 1
The generation of the DRL-based motion planning approaches.

	1st generation	2nd generation	3rd generation
Training task	Single target	Multiple target (restricted area)	Multiple target
Scalability	X	Δ	O
Training difficulty	Low	Intermediate	Challenging
Reference	Yang and Wang (2022a,b), Wen et al. (2018)	Li et al. (2020), Yang et al. (2021), Tang et al. (2022)	Man et al. (2021), Ni et al. (2021)

plan the motion for target points of a limited range. However, these approaches face limitations when expanding to the entire workspace of the robotic arm. This is because target positions in its workspace have different levels of difficulty for the robotic arm to reach and each level has causality with other levels. The existing approaches have limitations in considering the context of these difficulty levels (Saglietti et al., 2022). This limitation can cause the AI model to fall into a local optimum without fully comprehending the objective function for the motion planning.

In this paper, we propose a guided deep reinforcement learning (G-DRL) framework to address the training instability caused by the heterogeneous difficulty levels of target points within the entire workspace. The proposed framework includes three phases: (1) generating units of curriculum (UoCs) by clustering points with similar difficulty levels in the entire workspace, (2) determining the relative difficulty level of each UoC, and (3) sequentially training the DRL agent according to the curriculum, which consists of the generated UoCs and their determined difficulty levels. In other words, the curriculum guides the DRL agent to achieve an optimal solution on the motion planning task across the entire workspace.

The rest of this paper is organized as follows: Section 2 analyzes the existing AI-driven approaches for the motion planning of the robotic arm and specifies our contribution in comparison to them. Section 3 provides the background information of the deep reinforcement learning (DRL) algorithm. Section 4 describes the proposed G-DRL framework comprising three modules. Section 5 details the experimental settings for designing the curriculum and training the DRL agent. Section 6 presents the experiments based on the designed settings. Finally, Section 7 concludes this paper.

2. Related works

2.1. Deep reinforcement learning-based motion planning

Generally, traditional physics-based motion planning approaches, which utilize high dimensional numerical analysis equations, are well-suited for planning the motions. However, these approaches can struggle to handle the complexity of nonlinear systems and are challenged by the extreme computing resources. In contrast, data-driven approaches, which model the nonlinear relationships between inputs and outputs based on data, are effective in addressing the issues of nonlinear systems (Tang and Hauser, 2017). Particularly, the DRL algorithm has garnered attention for robotic arm motion planning as it enables self-learning through trial-and-errors in given environments, thereby deriving optimal solutions for complex problems with relatively low resources (Guan, 2020). This paper categorizes existing DRL-based motion planning research into three generations according to the training tasks, as illustrated in Table 1.

The main challenge of the initial studies was to replace the physics-based motion planner with the DRL-based motion planner. Accordingly, initial studies mainly trained DRL agents to have motion planning ability for the fixed target point, aiming to generate optimal motion plan from the initial state to the target point (Yang and Wang, 2022a,b; Wen et al., 2018). To this end, they focused on modeling the Markov decision process (MDP) and applying DRL algorithms such as deep deterministic policy gradient (DDPG) and proximal policy optimization

(PPO). They received a lot of attention by demonstrating that motion planning can be performed with the DRL. However, these studies were limited to the single target point only, resulting in an inability to generate optimal paths for unlearned target points. This limitation indicates the initial studies cannot be scalable to new target points.

In fields such as manufacturing and logistics industries, it is essential for robotic arms to reach various target points. However, extending the range that the DRL agent will learn to plan the motion indicates enlarging the exploration space. If the space that the agent must explore increases while the MDP model remains fixed, a problem arises where the reward function becomes sparse. The sparse reward makes it highly unlikely for the agent to receive meaningful rewards, thereby limiting its exploration. To address this limitation, subsequent studies focused on elaborately designing the MDP such as continuous reward functions and applying the efficient experience utilization methods such as the hindsight experience replay (HER) (Li et al., 2020; Yang et al., 2021; Tang et al., 2022). While these studies advanced by generating optimal paths for multiple target points, they were still limited to a portion of the workspace, making it challenging to extend to points not included in the training.

Recent studies aimed to develop the DRL agent to perform motion planning for target points across the entire workspace. They found a way to expand the workspace by considering external factors of the DRL algorithm (Man et al., 2021; Ni et al., 2021). They researched how to configure each joint of the robotic arm and how to apply a bio-inspired system to the DRL-based motion planner to reach target points in the entire workspace. Ni et al. (2021) analyzed the impact of different combinations of robotic arm's joints on performance and utilized the DDPG algorithm to train the motion planning task for random points within the workspace. Man et al. (2021) enhanced the learning efficiency of DRL agents by developing a directional-driven reward system inspired by human brain mechanisms, applying it to a 3-DoF robotic arm with demonstrable success. These studies significantly improved scalability by successfully training the motion planning task for multiple target points within the workspace.

Meanwhile, the relationship between the workspace and the complex dynamics of the robotic arm inevitably creates variations in difficulty levels for reaching target points. For instance, traditional physics-based approaches also experience increased computational complexity when calculating solutions for points within a complex space, compared to a simple space. Without considering the causality of these difficulty variations, the DRL agent might randomly learn easy and difficult points in each episode, which complicates the understanding of the objective function for the changing tasks (Saglietti et al., 2022). This could lead to overfitting on easier difficulty levels that the agents can readily handle. Since existing methodologies cannot consider this context, they face limitations such as limited performance for difficult target points or excessive training time, even when learning the entire workspace.

This paper aims to address these limitations by developing a curriculum that can account for the causality of heterogeneous difficulty levels among target points. The proposed framework creates the UoCs by clustering the target points with similar difficulty levels and determines their relative learning sequence. Subsequently, the DRL agent is trained using the developed curriculum, progressing from lower to higher difficulty levels sequentially. This approach incorporates the context

of motion planning difficulty for each target point, not considered in previous methodologies, to the DRL algorithm. It allows for more consistent performance across all difficulty levels with the advantage of reduced training time.

2.2. Curriculum learning

A curriculum is a systematically organized plan that defines the learning objectives, the learning contents, and the learning sequence of the human educational process. This ensures effective learning through balanced education and promotes the overall growth of students. A representative example of a curriculum can be found in mathematics. For instance, understanding equations can be set as a learning objective, with algebra, inequalities, linear equations, and quadratic equations as the learning content, and their order as the learning sequence.

The curriculum learning is an AI model training method inspired by the curricula used in the human educational learning process. The curriculum learning systematically sorts the training data according to the task's difficulty and progressively presents the data from simple tasks to challenging tasks to the AI model so that it can stably understand the learning objectives. The advantage of the curriculum learning is that it can reduce learning instability that can occur when learning complex tasks from scratch, and the agent's learning time can be shortened by understanding the relationship between simple tasks and challenging tasks. However, curriculum learning also has the disadvantage that it is difficult to design a curriculum for training AI models. If the curriculum is designed incorrectly, learning performance may deteriorate, and learning may become unstable.

In the early stages of research, the curriculum learning was mainly applied to supervised learning domains because learning data was available (Soviany et al., 2022). Recently, it has been extensively applied to DRL domains to effectively train autonomous robots that optimally control their dynamics according to the given state. In terms of robotic arm motion planning, curriculum learning approaches are already utilized to enhance both the accuracy of the reaching task and the learning stability (Zhou et al., 2021b,a; Kerzel et al., 2018; Or et al., 2023).

For example, Kerzel et al. (2018) designs a task simplification approach that gradually adjusts the size of the reaching target during the learning process. They set the random reaching accuracy requirements loosely when starting the training to allow the DRL agent to easily acquire initial knowledge, and gradually strengthen the requirements in later stages. Through this approach, they can plan the motion for the entire workspace. However, this approach is difficult to apply to current robotic arms with a high DoF because it targets a 2-DoF robotic arm.

Zhou et al. (2021a) proposes an obstacle differentiation approach that gradually adjusts the size of the obstacles according to the agent's performance. They start by setting a small cylindrical obstacle. When the agent completes each level of the curriculum, the size of the obstacle is increased. Finally, the trained agent can plan the motion to avoid the largest obstacle. However, this approach can only plan the motion from the fixed starting point to the fixed destination point. Therefore, the trained agent cannot provide scalable access to multiple target points.

Zhou et al. (2021b) proposes a residual reinforcement learning method that divides the training step into two: initial policy and residual policy. The initial policy is utilized to guide the training direction of the residual policy. It learns more easier task than residual policy. Then, the residual policy is trained to complement the shortcomings of the initial policy or to work in conjunction with the initial policy. However, they heuristically consider the learning contents (e.g., target points), indicating that it can cause a huge amount of time and cost to develop the curriculum, and the DRL agent's learning with the curriculum can become unstable.

Or et al. (2023) devises a step-by-step task learning approach that gradually adjusts the training objectives upon finishing each step's task. It mainly consists of three lessons. The first lesson teaches the position control task from the fixed starting point to the fixed target point. The second lesson teaches the position and posture control tasks from the fixed starting point to the fixed target point. The final lesson teaches flexible movement from the fixed starting point to the fixed target point. Through this process, the agent can gain flexible manipulation abilities in limited areas. However, the robotic arm's workspace is limited to specific areas, similar to Zhou et al. (2021a). If the exploration space of the DRL agent is extended to the entire workspace, it is easy for the agent to get lost in the swamp of exploration. This makes the agent fall into the easy task preference problem or over-fitting problem so that the agent cannot learn the motion planning task in the entire workspace.

Matiisen et al. (2019) devises a teacher-student curriculum learning method that divides the roles into two actors. The student learns complex tasks using a policy network, while the teacher automatically selects the student's learning tasks by monitoring its learning curve. Additionally, this paper integrates a uniform sampling mechanism to address the catastrophic forgetting problem that occurs during the teacher-student curriculum learning. It demonstrates the potential of automatic curriculum learning in tasks such as the addition of decimal numbers and navigation in Minecraft. However, this paper can suffer from the performance degradation because they cannot consider the relationship of the sub-tasks.

2.3. Main contributions

The main contributions of this paper can be summarized as follows:

1. A curriculum generation method is designed to effectively train the DRL agent for the multi-point motion planning task in large-scale exploration spaces. The generated curriculum guides the DRL agent to consider the causality arising from different difficulty levels of target points across the entire workspace.
2. To mitigate the catastrophic forgetting problem, where the DRL agent forgets the policies for previously learned lower-level UoCs while training on higher-level ones, a review mechanism is introduced. This mechanism enables the DRL agent to consistently plan motion for target points with different difficulty levels.
3. To address the data imbalance problem that occurs when integrating the curriculum with the vanilla DRL algorithm, a buffer flushing mechanism is applied to the replay memory of the DRL agent. This mechanism ensures the ideal data distribution by maintaining a balanced mix of previously learned tasks and the currently learning task, thereby facilitating the review mechanism more effectively.
4. Extensive experimental results demonstrate that the proposed framework outperforms both the baseline DRL algorithm and the state-of-the-art motion planning methods in terms of performance and speed for training the multi-point motion planning task.

3. Background

3.1. Markov decision process

The MDP is a modeling framework used for decision-making problems, designed to derive the optimal strategy over time in a probabilistic environment. In general, an MDP consists of a state space S , an action space A , a reward function $R(s, a)$, a state transition probability $P(s'|s, a)$, and a discount factor γ . The variables s and s' denote the current and next state, respectively, while a signifies the action that the agent can take in a given state (Altman, 2021; Li, 2017).

The state space S is the set of potential environmental observations that the agent can consider when taking an action, while the action space A denotes the set of possible actions that the agent can take. The reward function $R(s, a)$ provides the expected reward to the agent when an action a is taken in a specific state s . The state transition probability $P(s'|s, a)$ represents the probability of moving to the next state s' when an action a is taken in the current state s , modeling the uncertainties and dynamic changes in the environment. The discount factor γ , ranging from 0 to 1, is a coefficient used to convert future rewards into their present value. It determines how much long-term rewards will be considered compared to short-term rewards. If the discount factor is close to 1, the agent places more emphasis on long-term rewards.

The main objective of an MDP is to find the optimal policy π^* , which is a function that performs the best action in each given state. The optimal policy outputs the actions of the agent to maximize the expected cumulative rewards as shown in Eq. (1).

$$V^{\pi^*}(s) = \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s \right] \quad (1)$$

$V^{\pi^*}(s)$ represents the state-value function, reflecting the total expected cumulative rewards under a given state s and the optimal policy π^* . By evaluating the value of each state through this function, the agent can identify which states will bring more rewards in the long run and the policy is optimized according to the given state-value function.

3.2. Deep reinforcement learning algorithm

The DRL algorithm is a contemporary approach to solve decision-making problems in environments with high-dimensional state space, integrating the neural networks with traditional reinforcement learning (RL) methods. The key components of the DRL algorithm include the environment and the agent, which interact with each other. When training the agent, it observes the state from the environment and makes an action based on the observed state. The environment reflects the agent's action at time step t and provides a reward to the agent. The primary goal of the DRL algorithm is to learn the agent's optimal policy π^* that maximizes the total rewards in a given environment (Basile et al., 2024).

The classification of the DRL algorithm, which trains the networks within the agent, is divided into Q-learning based and policy gradient based approaches, each adopting different strategies to learn the optimal policy. In the Q-learning based approach, the agent learns to predict the Q-values for every possible action a in each state s . The Q-value represents the expected cumulative reward for taking a specific action in the environment. The objective of the Q-learning based approach is to find optimal action-value function $Q^*(s, a)$ through the Q-value update equation, as shown in Eq. (2); α is the learning rate.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (2)$$

On the other hand, the policy gradient based approach trains the neural network to directly learn the optimal policy $\pi_{\theta}(a|s)$, modeling the probability of taking action a under a given state s in the environment. The goal in the policy gradient based approach is to maximize the expected reward, as shown in Eq. (3). This approach encourages the agent to choose actions more frequently that are likely to yield higher rewards.

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) R(s, a) \right] \quad (3)$$

3.3. Soft actor-critic

The SAC algorithm utilizes a Gaussian policy to model the probability distribution of actions when given a state, allowing for efficient policy learning in environments with continuous action spaces. By leveraging entropy to optimize the balance between exploration and exploitation, it can perform effective learning in relatively high-dimensional environments compared to other DRL algorithms (Haarnoja et al., 2018).

Despite the advantages of the SAC algorithm, it still has limitations in solving the high-dimensional problems, particularly when training the motion planning task for multiple points within the entire workspace of a robotic arm. This is because of the easy task preference problem, which concentrates on the states and the actions with the known rewards. To demonstrate and address this limitation, we set the SAC algorithm as a baseline and apply the devised curriculum to the baseline algorithm, developing the G-DRL framework. Then, we compare the performance of the baseline algorithm and the G-DRL framework. We describe the key components of the G-DRL framework in Section 4 and verify its novelty in Section 6.

4. The guided deep reinforcement learning framework

This section provides a description of the proposed G-DRL framework which applies the automated curriculum scheme (ACS) to accurately plan the motions for multiple target points within the robotic arm's workspace. As shown in Fig. 1, the proposed framework is divided into three modules: data engineering module, curriculum building module, and task training module.

4.1. Data engineering module

The data engineering module generates and pre-processes the features for building the curriculum. This module mainly consists of the feature generator and feature selector.

4.1.1. Feature generator

To build the curriculum, the feature generator creates feature data for each target point within the entire workspace that the robotic arm can reach. The overall workflow of the feature generator is shown in Fig. 2. First, the feature generator calculates the workspace of the target robotic arm, considering the constraints such as the length of each link and the angle limitations of each joint. As shown in Fig. 2(a), it then uniformly extracts the points that the end effector of the robotic arm can reach. Finally, it derives the feature data for the extracted points using the inverse kinematics solver, which translates the desired position of the end effector into the specific joint angles (Lopez-Franco et al., 2018). If feature data have different scales, clustering can be biased towards specific features with larger magnitude, potentially skewing the curriculum generation process. Therefore, it normalizes output data for each feature.

The features created through the feature generator can consist of four baseline data as shown in Fig. 2(b): orientation, distance, run time, and delta. The orientation feature indicates the Euler angle between the initial state of the end effector and the target point in the same coordinate system. The distance feature represents the Euclidean distance from the initial position of the end effector to the target point. In general, the closer the distance, the fewer moves it takes to get there, and the farther the distance, the more moves it takes to get there. This phenomenon justifies that the distance can be related to difficulty. The delta feature, calculated by the inverse kinematics solver, signifies the moving angle of each joint to reach the target point. Similar to the distance feature, the less the angle changes, the easier it is to plan the motion. However, in certain cases, there may be a large delta value despite a small distance. This suggests that these two features, distance and delta, may have similar but complementary

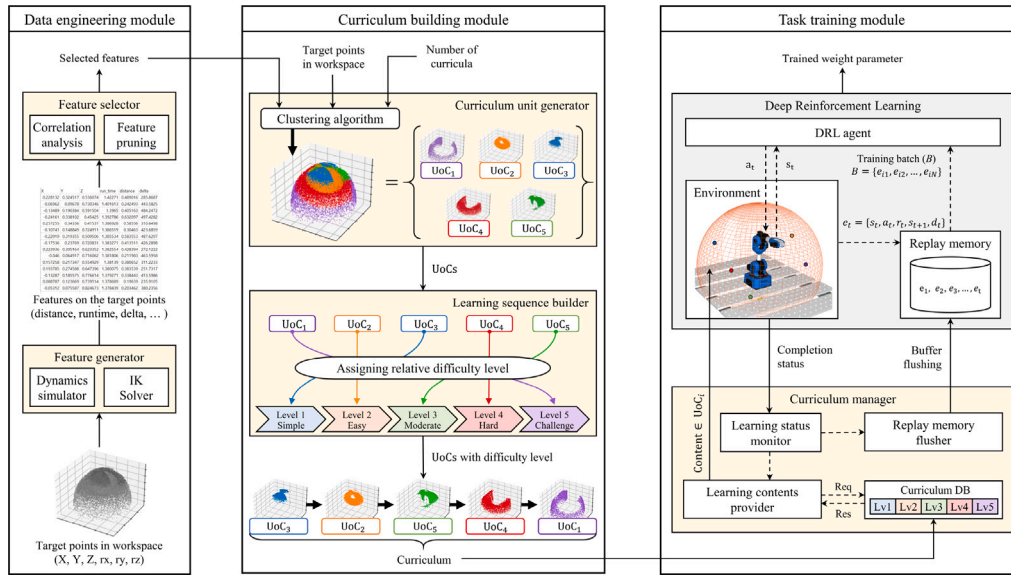


Fig. 1. The guided deep reinforcement learning framework for accurately planning motions on multiple target points within the workspace; gray blocks indicate the legacy systems.

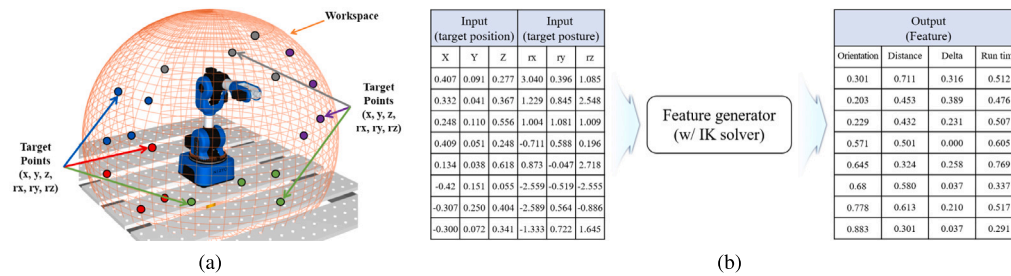


Fig. 2. (a) Examples of the target points and the entire workspace; the target points are uniformly extracted within the reach of the robotic arm; (b) examples of the input and output of the feature generator; this outputs features that can express the difficulty of reaching the target point.

properties. In other words, the delta feature may capture properties that the distance feature does not. Finally, the run time feature means the processing time that it takes for the inverse kinematics solver to compute a solution to the target point. The computational complexity of the inverse kinematics solver depends on the constraints of the robotic arm and the properties of the target point, which is reflected in the processing time.

Features not considered above can also be added to the framework for curriculum generation. If they have characteristics similar to other features, they will be automatically filtered out by the feature selector. On the other hand, if they have new characteristics, they can be used as new features.

4.1.2. Feature selector

The feature selector pre-processes the derived features to generate the curriculum under optimal conditions. If engineers empirically add various features and create UoCs (clusters) based on them, unnecessary features may act as noise and degrade clustering performance. In order to complement this issue, the feature selector identifies and selects the non-overlapping and useful features from the set of derived feature data. In other words, it performs pruning on the features to ensure that the AI model does not consider unnecessary ones when generating UoCs. This process helps to divide the UoCs into more logical segments.

To this end, the feature selector collects the derived features from the feature generator. Then, it creates a correlation matrix among the features (Waskom, 2021). The elements of the correlation matrix represent linear correlations between the features and can have a value

from -1 to 1 . The closer its absolute value is to 1 , the more highly correlated the features are, and the closer it is to 0 , the less correlated they are. If the correlation value between two features is greater than 0.8 , the feature selector prunes the feature that is more correlated with other variables. Otherwise, it uses all features for generating the UoCs without pruning.

4.2. Curriculum building module

A curriculum is a plan that organizes the learning contents and learning sequence in a particular subject area, arranged from easier to harder contents to help learners acquire knowledge effectively. In this context, the subject area is the multiple point motion planning task within the entire workspace, the learning contents are the UoCs, and the learning sequence is ordered by the difficulty levels of the UoCs. To realize this definition, the curriculum building module automatically generates the UoCs without human intervention by clustering the target points with similar motion planning difficulty level within the workspace. Then, it assigns the relative difficulty levels to the generated UoCs. This module consists of the curriculum unit generator and the learning sequence builder.

4.2.1. Curriculum unit generator

The curriculum unit generator receives the selected features and performs clustering based on multiple unsupervised learning algorithms to generate candidates for UoCs. To select the best UoCs that are logically well separated from each other, the candidates are validated

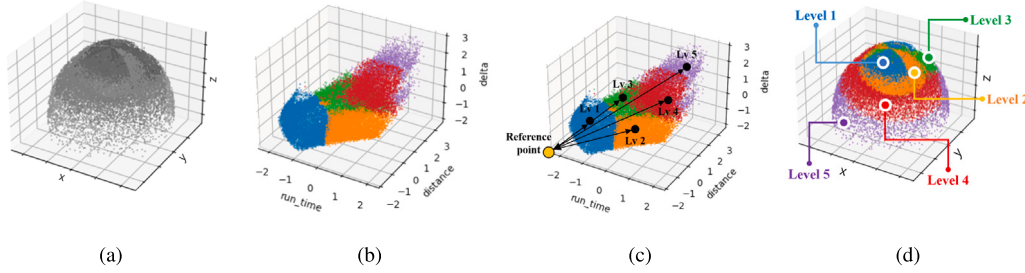


Fig. 3. (a) The extracted target points within the workspace, (b) the example of the generated units of curriculum based on the clustering algorithm in the feature vector, (c) the example of the relative difficulty level assignment based on feature vector distance, and (d) the example of the projected results of the assigned difficulty levels.

using metrics that evaluate the quality of the clustering results. The best candidate is extracted from the validation results as shown in Fig. 3(b), and is sent to the learning sequence builder.

To select the best candidate for the UoCs, we used four evaluation metrics for clustering: Silhouette, Calinski–Harabasz, Dunn index, and Davies–Bouldin. These metrics assess how closely data are clustered within a cluster and how well they are separated from other clusters (Shahapure and Nicholas, 2020; Wang et al., 2021; Singh et al., 2020; Rivera-Borroto et al., 2012). The Silhouette metric measures the similarity to other clusters and calculates the Silhouette coefficient. The coefficient ranges from -1 to 1 and is closer to 1 when the clustering quality is high. The Calinski–Harabasz metric calculates its coefficient by dividing the inter-cluster variance by the intra-cluster variance. The higher values of the coefficient indicate better clustering quality. The Dunn index considers the intra-cluster compactness and inter-cluster separation. It calculates the distance between the heterogeneous clusters and the maximum distance between data points within clusters. This index also has the characteristic that the larger the value, the better the clustering quality. The Davies–Bouldin metric calculates the ratio of the internal variance of a cluster to the separation between clusters. The property of the calculated coefficient differs from other evaluation metrics, with lower values indicating higher quality. This validation process is crucial because different UoCs might have similar properties if not appropriately separated, potentially causing confusion for the DRL agent when learning the curriculum.

4.2.2. Learning sequence builder

The learning sequence builder determines the relative difficulty levels on the generated UoCs. As shown in Fig. 3(b), it is hard to identify whether each UoC has a relatively high or low difficulty level, respectively.

To map the level of difficulty to each UoC, the learning sequence builder identifies the center points of each UoC. It then sets the reference point at the lowest values of each feature in the feature vector as shown in the orange dot in Fig. 3(c). The learning sequence builder calculates the feature vector distances between the reference point and the center points of the UoCs. It sorts the difficulty levels of UoCs by the calculated feature vector distance values, as shown in Fig. 3(c). The UoCs with a short feature vector distance from the reference point are considered less challenging, while those with a relatively long feature vector distance are evaluated as more challenging. Fig. 3(d) shows the projected results of each UoC within the workspace of the robotic arm.

The curriculum built through this process systematically guides the learning of motion planning tasks for multiple points within the workspace. Each step of the curriculum has a clearly defined training goal and a difficulty level, leading the agent to understand the context of the curriculum during the learning process of the DRL agent. The agent learns tasks sequentially from easy to difficult levels, ultimately enabling it to plan motions across a full range of the workspace. In other words, the curriculum helps the agent understand its objective function.

4.3. Task training module

Before understanding how the task training module operates, it is essential to identify the core mechanisms: review and buffer flushing.

A curriculum scheme that continues to learn new contents based on the policy trained with the previous contents can lead to a catastrophic forgetting problem. It indicates the phenomenon that knowledge of the previously learned task is abruptly lost upon integrating new knowledge of currently learning task (Kirkpatrick et al., 2017; Kanitscheider et al., 2021).

In this paper, the review mechanism is devised to effectively mitigate this problem. As shown in Fig. 4, this mechanism allows the agent to simultaneously learn the contents it is currently learning (e.g., target points of the level 3) and the contents it has previously learned (e.g., target points of the levels 1~2), even as it transitions to a new learning phase (e.g., level 3). The devised mechanism not only ensures that the agent does not forget what it has previously experienced, but also gives it the ability to understand the correlation between previous and current difficulty levels, making the acquired knowledge more robust.

The replay ratio is a hyper-parameter that determines the selection rate of previously learned contents in the review mechanism. If the replay ratio is 0.2 , there is a 20% chance that the previously learned contents will be recalled and an 80% chance that the currently learning content will be loaded, for every episode. The larger the value, the more the agent will perform the review, and the smaller the value, the less it will review.

Shortly after the curriculum scheme transfers the training target of the DRL agent to a new difficulty level, the replay memory of the DRL agent is composed of both the previously learned data and the currently learning data of the new level. The previously learned data occupies a large portion of the replay memory, while the currently learning data is comparatively small. This means that the ratio between the data of the previously learned tasks and the data of the currently learning task is significantly unbalanced. Their ratio will gradually become balanced after the data of the currently learning task accumulate in the replay memory over time. This can cause the agent to be unable to prioritize training for the currently learning task, which will considerably delay the training time.

To solve this problem, a buffer flushing mechanism for the replay memory is also devised. The devised mechanism clears the buffer of the replay memory whenever the agent finishes learning the task on the current difficulty level, allowing the agent to focus on training at the next level. In addition, it supports the DRL agent to learn the task with an ideal data distribution, configured based on the replay ratio.

Buffer flushing is a hyper-parameter with the property of a flag. It determines whether the buffer flushing mechanism is applied to the replay memory when training the DRL agent. If the value is true, the agent uses the buffer flushing mechanism. Otherwise, the agent does not use the mechanism during curriculum-based training (see Fig. 5).

The next subsection describes the components of the task training module, which trains the DRL agent to plan the motion for multiple points with the generated curriculum; the curriculum manager and the DRL agent.

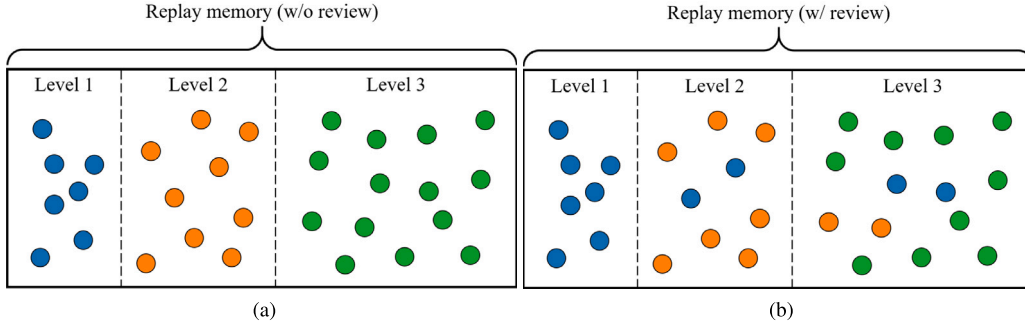


Fig. 4. (a) The example on the replay memory of the DRL agent without considering the devised review mechanism, (b) the example on the replay memory of the DRL agent with considering the devised review mechanism; in the figures, blue point means the trajectories (state $s_{0:n}$, action $a_{0:n}$, reward $r_{0:n}$, next state $s'_{0:n}$) at one episode of the difficulty level 1; the orange point represents the data at the level 2 and the green point indicates the data at the level 3.

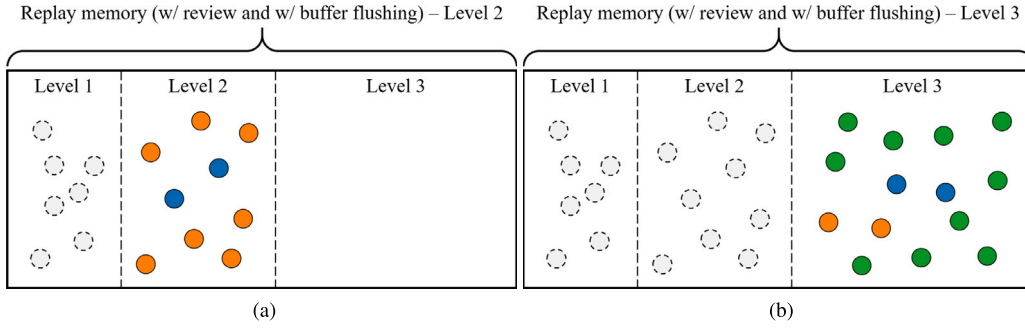


Fig. 5. The example on the replay memory of DRL agent with considering the devised buffer flushing mechanism; (a) when the difficulty level of the curriculum is transferred from the level 1 to the level 2, (b) when the difficulty level of the curriculum is transferred from the level 2 to the level 3; if the buffer flushing mechanism is not applied in case (b), the ratio of the level 1 and 2 data overwhelms the ratio of the level 3 data, initially.

4.3.1. Curriculum manager

The curriculum manager guides the learning process of the DRL agent to train the multi-point motion planning task according to the generated curriculum. It operates as an add-on module for the legacy DRL framework, and is interfaced with both the DRL environment and replay memory. The functional blocks of the curriculum manager can be summarized as follows:

(1) Learning contents provider

Before providing the learning content (i.e., target point for training the motion planning task), it identifies the set replay ratio and determines the difficulty level of the content that the DRL agent will learn according to the ratio. Then, it requests the learning content from the curriculum database according to the determined difficulty level. Finally, it receives the learning content and transmits it to the DRL environment. If the DRL agent completes the training at the set difficulty level, the learning content provider increases currently learning difficulty level. Otherwise, it maintains the difficulty level of the curriculum.

(2) Learning status monitor

It identifies the completion status of the DRL agent in order to notify the learning contents provider and replay memory flusher. It conveys the success rate at the currently learning difficulty level and episode finish information to the learning contents provider. Then, the learning contents provider transmits the prepared new learning content to the DRL environment. The learning status monitor also serves the success rate to the replay memory flusher so that it can clean the accumulated replay memory of the DRL agent.

(3) Replay memory flusher

When the replay memory flusher receives the success rate of the DRL agent from the learning status monitor, it identifies whether the

pre-defined flushing conditions are met. If the success rate exceeds the difficulty level transfer condition, it flushes the replay memory of the DRL agent. Through flushing, the DRL agent becomes more sensitive to the new learning contents at the next difficult level.

4.3.2. Deep reinforcement learning agent

The DRL agent learns the motion planning task on the multiple target points through collaboration with the curriculum manager. The DRL agent follows an MDP to learn the optimal policy for the task. The designed MDP is as follows:

(1) State space

To perform the motion planning task for multi-target points within the defined workspace, the DRL agent should know three categories of data to recognize the situation and control the robotic arm's joints: (1) the angle values of each joint (6-axis) $j_n(t)$, (2) the position of the end-effector $p_e(t)$, and (3) the position of the goal point $p_g(t)$. Therefore, the state space of the agent s_t can be defined as follows:

$$s_t = [j_{1-6}(t), p_e(t), p_g(t)]^T \quad (4)$$

Where t is the time step during the training of the DRL agent. The position data consist of three-dimensional position data (x, y, z). The data of the state space is used as input to the DRL agent's neural network, which is trained to output the best action for the given state.

(2) Action space

To perform motion planning from the current state to the target point, the DRL agent should rotate the joints related to the reaching task. Therefore, the action space of the DRL agent a_t can be defined as controlling three joints as shown in Eq. (5): the base, shoulder, and elbow.

$$a_t = [a_{base}, a_{shoulder}, a_{elbow}], \quad \forall a_t \in [-\beta_{base\sim elbow}, +\beta_{base\sim elbow}] \quad (5)$$

Since the control of the robotic arm has a continuous characteristic, the DRL agent's action space should be designed to output continuous values. To this end, we define the boundaries of each action space ($\beta_{base \sim elbow}$) as shown in Eq. (6).

$$\beta = |\text{upper joint limit} - \text{lower joint limit}| / \delta \quad (6)$$

The upper and lower joint limits are the values to which each joint can rotate. These values are usually specified in the robot's specification. Subtracting the lower limit from the upper limit for each joint allows the calculation of the range within which each joint can move. δ is a hyper-parameter that determines the amount of the control at every time step. If δ has a higher value, the boundary of the agent's action will be narrower. Otherwise, the boundary will be broader.

The DRL agent outputs an action value within the designed boundary at each time step, making the robotic arm rotate each joint by the output value in its current state. If the action value is lower, the agent rotates the joints in small increments at a single time step. Otherwise, the agent may rotate the joints in large increments. The DRL agent's action value is determined by the output of the actor network.

(3) Reward function

The reward function defines what actions the agent should take to achieve the desired goal. The goal of the motion planning is to reach multiple changing target points without colliding with itself. For this purpose, the reward function of the agent r_t is designed as shown in Eq. (9):

$$r_t = r_{ds}(t) + r_{ss}(t) + r_{cl}(t) + r_{st}(t) + r_{dv}(t) \quad (7)$$

The reward function is divided into five sub-reward functions in terms of the distance, success, collision, max step, and divergence. The reward function for the distance operates at every time step during the training phase, while other episodic reward functions work in an event-driven manner. If the episodic reward functions provide a negative or positive reward to the agent, the training episode is terminated.

To train the DRL agent on the motion planning task, the agent should receive a higher reward as the end-effector approaches to the target point. In this context, a distance-dependent reward function is designed as detailed in Eq. (8).

$$r_{ds}(t) = -\sqrt{(x_e(t) - x_p(t))^2 + (y_e(t) - y_p(t))^2 + (z_e(t) - z_p(t))^2} \quad (8)$$

The reward function $r_{ds}(t)$ calculates the Euclidean distance between the target position and the end-effector's position at every time step t . Subsequently, the distance is multiplied by a negative value, thus providing a lower reward for a greater distance and a higher reward for a shorter distance to the agent.

In order for the agent to learn how to accurately move the end-effector to the target point, it is essential to provide a positive episodic reward when the end-effector successfully arrives at the point. If the agent only learns from distance-dependent reward information, the end-effector may oscillate near the target point without actually reaching it. Accordingly, a reward function for the success $r_{ss}(t)$, providing a reward of a positive constant S upon reaching the goal, is formulated as follows:

$$r_{ss}(t) = S + CL \text{ (if the end effector reaches the target point)}, 0 \text{ (else)} \quad (9)$$

If the reward function $r_{ds}(t)$ only includes the information for success, the agent cannot understand the context of changing difficulty levels in the curriculum scheme. To this end, we design a variable

CL representing curriculum levels and incorporate it into the reward function $r_{ss}(t)$. The variable CL is designed as shown in Eq. (10):

$$CL = k * (\text{difficulty level of the curriculum} - 1) \quad (10)$$

Where k is the hyper-parameter that determines the amount of the reward. The variable CL adjusts the success reward value according to the difficulty level during the curriculum learning process. When the end effector of the robotic arm reaches the target point, the DRL environment identifies the difficulty level of the UoC set by the curriculum manager and provides a reward accordingly. For example, if the curriculum manager sets the target point in the UoC with difficulty level 2, the variable CL has the value $1 * k$ value and it is added to the constant S . This helps the DRL agent to understand the curriculum.

While the agent is performing motion planning, it may cause an accident if it collides with itself. To prevent this, the reward function $r_{cl}(t)$ is designed as follows:

$$r_{cl}(t) = C \text{ (if collision} == \text{True)}, 0 \text{ (else)} \quad (11)$$

In the reward function $r_{cl}(t)$, the constant C is a negative value. In other words, the reward function $r_{cl}(t)$ provides a negative reward to the agent when it collides. Therefore, the agent learns to avoid collisions with itself.

When performing motion planning, the end effector needs to reach the target point in a short time, meaning that the DRL agent can make an optimal solution for the target position. If the robot is constantly oscillating in a place that is not related to the target point, it is not an optimal result. To this end, the reward function $r_{st}(t)$ is devised as follows:

$$r_{st}(t) = C \text{ (if the training step in the episode} > 256), 0 \text{ (else)} \quad (12)$$

If the number of training steps exceeds 256 in an episode, the reward function assumes that the agent cannot perform optimal control to the target point and provides a negative episodic reward C to the agent. Therefore, the agent learns how to optimize the motion planning to reach the target point in a short time.

Finally, we design the reward function to help the agent learn quickly by reducing unnecessary trial-and-errors. The designed reward function for divergence $r_{dv}(t)$ is shown in Eq. (13):

$$r_{dv}(t) = C \text{ (if } -r_{ds}(t) \geq z \text{ cm)}, 0 \text{ (else)} \quad (13)$$

If the agent controls each joint to move away from the target point up to a certain distance (z), the episode of the agent is terminated and the reward function gives the agent a negative episodic reward. This induces the agent to avoid experiencing bad trial-and-errors, unrelated to motion planning on the target point. The negative episodic reward C in the reward functions $r_{cl}(t)$, $r_{st}(t)$, and $r_{dv}(t)$ is a hyper-parameter and has the same value.

During training, the episode of the DRL agent is terminated when the end-effector of the robotic arm accurately reaches the target position within a 1 cm margin of error or the agent receives the negative reward from the sub-reward functions. If the end-effector reaches the target point, the success flag for that episode is raised.

The DRL environment shares the termination results at each episode with the curriculum manager. If the episode terminates and the success rate of the last 10 episodes does not exceed the curriculum transition condition, the curriculum manager re-configures the target point for motion planning according to the current difficulty level. If the episode is terminated and the success rate of the last 10 episodes exceeds the condition, the curriculum manager increases the difficulty level of the curriculum. If the DRL agent finishes the training at the final level of the curriculum, it outputs the trained policy and terminates the training process.

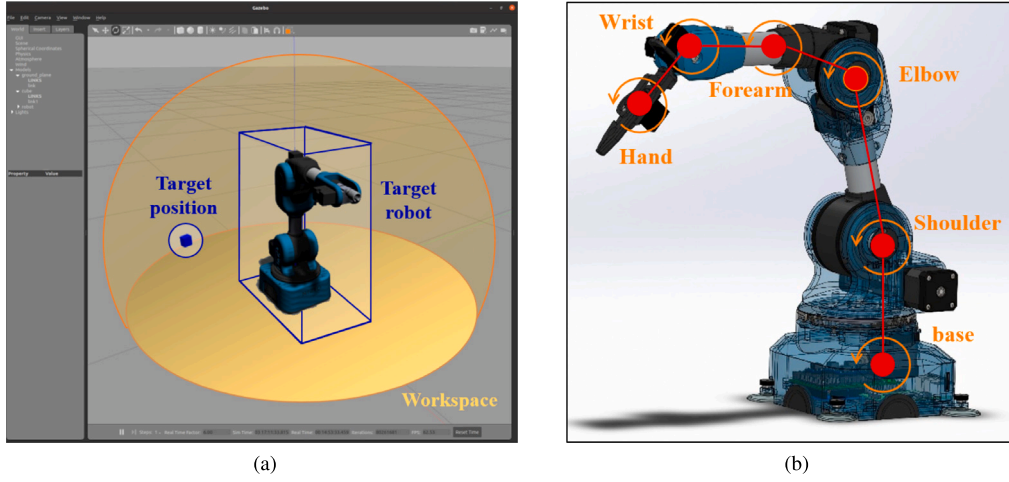


Fig. 6. (a) The experimental environment constructed based on the Gazebo simulator and unified robot description format, (b) the robotic arm used in the experiment (Niryo Ned2); it mainly consists of 7 links (base, shoulder, arm, elbow, forearm, wrist, hand) and 6 joints (base, shoulder, elbow, forearm, wrist, hand), meaning it has 6 degree of freedom.

Table 2

The properties of links and joints of the Niryo Ned2; the units of the link length and joint angles are mm and degree ($^{\circ}$), respectively.

Properties	Base	Shoulder	Arm	Elbow	Forearm	Wrist	Hand
Link length	95	90	235	78	312	111	99
Joint angle	(-169, 169)	(-120, 35)	-	(-77, 90)	(-120, 120)	(-110, 110)	(-145, 145)

Table 3

The experimental factors for generating the units of curriculum with the extracted features.

Experimental factors	Details
Clustering algorithms	K-means, Bisecting K-means, Mini-Batch K-means, Gaussian Mixture, Bayesian Gaussian Mixture
Quality verification metrics	Silhouette, Calinski-Harabasz, Davies-Bouldin, Dunn index
Number of UoCs	3, 5, 7

5. Experimental settings

5.1. Data engineering module

As shown in Fig. 6(a), we utilized the gazebo simulator (Agüero et al., 2015), which can simulate the physical dynamics of a robotic arm, to extract feature data for generating the curriculum. The Gazebo simulator is based on the unified robot description format (URDF) to accurately simulate the dynamics of the robotic arm by reflecting physical properties. The URDF, which utilizes an XML file format to describe a robot model, can define physical properties such as constraints, dynamics, and visual geometry for a robot (Kang et al., 2019).

To conduct experiments on curriculum generation and task training, we configured the URDF for the Niryo Ned2 robotic arm and spawned the simulation objects in the Gazebo simulator. The properties of the links and joints of the Niryo Ned2 described in the URDF are shown in Table 2. Additionally, we utilize the MoveIt framework (Coleman et al., 2014), which includes a component on the inverse kinematics, to extract the features on the orientation, distance, run time, and delta.

5.2. Curriculum building module

To experiment on generating the UoCs, we set the experimental factors as shown in Table 3. In this experiment, five clustering algorithms based on unsupervised learning were applied to generate the candidates for the UoCs and three metrics were used to verify the clustering quality of the generated candidates. The number of UoCs was set to 3, 5, and 7 to identify how a high or low number of UoCs affects the training of the DRL agent.

5.3. Task training module

The hyper-parameters for training the DRL agent consist of the part of the SAC algorithm and the automated curriculum scheme (ACS) as shown in Table 4. In the hyper-parameter settings of the SAC algorithm, the learning rate was lowered to increase the training stability, and the replay buffer size was set to 1,000,000 to accommodate a substantial volume of data. A high replay buffer size is not meaningful when the buffer flushing mechanism is applied, but it is valid when the mechanism is not used. The remaining hyper-parameters are similar to those in typical SAC algorithms. The constants C , S , and δ , which are related to the designed reward functions, were heuristically determined to be -10, 50, and 50, respectively.

For the hyper-parameters related to the ACS, the numbers of UoCs, handled by the curriculum manager to train the DRL agent, were set to 3, 5, and 7, respectively. These values are the same as the numbers of clusters created by the curriculum generator. The currently learning level of the agent is transferred to the next level when the success rate at that level is above 90% success rate during the last 10 episodes. The replay ratio was experimented with a wide range from 0 to 0.5, and buffer flushing was also experimented with True and False values. In addition, the value of the coefficient k , which allows the DRL agent to adaptively receive an episodic reward on the success according to the difficulty level, was set to 12.5. This value is S divided by 4 because if the number of UoCs is 5, the number of transitions in the curriculum is 4. If the number of UoCs is 6, the number of transitions is 5, so the coefficient of CL was set to 10.

Table 4

The hyper-parameters for training the DRL agent with generated curriculum.

Hyper-parameters	Details
SAC algorithm	
Learning rate	0.00015
Discount factor	0.99
Replay buffer size	1,000,000
Number of hidden layers	2
Number of hidden units per layers	64
Number of minibatch sample	4096
Optimizer	Adam
Activation function	Linear
C (Negative episodic reward)	-10
S (Positive episodic reward)	50
δ (Control factor)	50
ACS	
Number of UoCs	3, 5, 7
Curriculum transition condition	90% success rate 5 times in a row
Replay ratio	0 ~ 0.5
Buffer flushing	True/False
k (Coefficient on curriculum reward)	12.5

6. Experimental results

In this section, we only explain the experimental results for the number of UoCs equal to 5, which has the most strong impact during training the DRL agent. Other experimental results for different numbers of UoCs are available in [Appendix B](#).

6.1. Experiments on generating the optimal UoCs

Experiments were conducted to determine how well the five unsupervised learning algorithms generate UoC candidates within the workspace of the robotic arm in terms of cohesion and separation of clustering. The algorithms generate the UoC candidates using the four baseline features (i.e., orientation, distance, run time, and delta) created by the feature generator.

[Table 5](#) shows the clustering quality verification results. Higher values of the Silhouette metric, Calinski–Harabasz metric, and Dunn index and a lower value of the Davies–Bouldin metric indicate better performance. The values in parentheses are the results of normalization from the lowest to the highest value in each verification metric.

Overall, the K-means-based algorithms outperformed the Gaussian mixture algorithms on the four verification metrics in the experiment. The sum of the normalization values of the K-means algorithms ranked first, second, and third in each metric, while the Gaussian mixture algorithms ranked fourth and fifth in each metric. These results show the UoC candidates generated by the K-means algorithms perform better in terms of the cohesion and separation, indicating that the data points within an UoC are located close to each other and the UoCs are clearly separated from one another. If these are not clearly separated as shown in the results of Gaussian mixture algorithms, UoCs can have similar characteristics to others because their edges will overlap each other. This can cause confusion for the DRL agent in understanding the characteristics of the data represented by the UoC.

To extract the optimal UoCs for training the DRL agent, each verification result was normalized from 0 to 1 for each indicator. The UoCs generated by the algorithm with the highest sum of the normalization values for all metrics are determined to be optimal. In this experiment, the vanilla K-means algorithm achieved the highest ranking. Therefore, in the following experiments, we ordered the difficulty levels on the UoCs generated by the vanilla K-means algorithm and utilized them as the learning contents of the curriculum for training the DRL agent.

6.2. Experiments on ordering the relative difficulty levels

Experiments were conducted to develop the learning sequence on the UoCs generated by the vanilla K-means algorithm. To this end, we identified the center points of each UoC and extracted the feature vector distances between the reference point and these center points. [Table 6](#) shows information on each UoC and each calculated feature vector distance.

The colors specified in [Table 6](#) are the same as the colors of each UoC visualized in [Fig. 7](#). The learning sequence builder of the proposed G-DRL framework assigns the relative difficulty levels for each UoC in order of decreasing feature vector distance. Therefore, UoC 3, which has the highest Euclidean distance, was assigned the highest difficulty level, meaning the data points (i.e., motion planning target points) in UoC 3 are the most challenging in the entire curriculum. Meanwhile, UoC 5, which is closest to the reference point, was allocated to the lowest difficulty level. This UoC will be utilized as the first training data when the DRL agent learns the curriculum. [Fig. 7](#) shows each UoC assigned the difficulty levels. The DRL agent sequentially learns from the target points in [Fig. 7\(a\)](#) to those in [Fig. 7\(e\)](#).

If calculating the feature vector distances between the reference point and the center points of the UoCs is not a proper method for assigning the difficulty levels of the UoCs, the training of the DRL agent with the curriculum may have poor performance because it cannot understand the causality arising from the difficulty levels between different UoCs. In the experiments of the next subsection, it is demonstrated that the difficulty level assigning method with the feature vector distance was very effective in training the DRL agent.

6.3. Experiments on training the DRL agent with the developed curriculum

Experiments were conducted to identify the performance of the designed ACS which includes review and buffer flushing mechanisms. we verify how the ACS well trains the DRL agent to plan the motion in the entire workspace, compared to the baseline SAC algorithm and recent works.

[Table 7](#) shows the verification results on success rates, training times, and training episodes when training the DRL agent on the motion planning task without and with the ACS. SAC w/o ACS refers to the baseline SAC algorithm which does not consider the proposed ACS. In the SAC w/ ACS, each number written after “shop” (e.g., # 0.5) indicates the replay ratios and the experiments are conducted with an extensive range of them from 0 to 0.5. In this experiment, the buffer flushing mechanism is also considered when applying the ACS. [Fig. 8](#) shows the success rates and accumulated rewards of the DRL agent trained based on the baseline SAC algorithm without the ACS. On the other hand, [Figs. 9](#) and [10](#) show the success rates and accumulated rewards of the DRL agents trained with the ACS according to different replay ratios.

As illustrated in [Table 7](#), the distribution of training times and episodes required for training each policy spans extensively, ranging from 25 to 40.9 h and from 9,113 to 38,629 episodes, respectively. Training the policy based on the baseline SAC algorithm consumes many episodes but, the aspect of the training time is similar to others. This phenomenon arises from the approach of setting the target point for the motion planning randomly in each episode, without accounting for the heterogeneous difficulty levels. Consequently, despite having a similar training time to others, the agent cannot fully understand the objective function for the motion planning task. This results in the agent frequently making collisions or divergences, causing the training episodes to terminate more quickly compared to other training conditions. In fact, the average success rate is less than 40%, even though it has been learning for a long time (episodes from 11,000 to 35,000) as shown in [Fig. 8\(a\)](#).

As shown in [Table 7](#), the policy derived from the baseline SAC algorithm successfully plans the motion for destination points in the

Table 5

The clustering quality verification results on the UoC candidates generated by the unsupervised algorithms; the values in parentheses are the results of normalization from the lowest to the highest value in each metric.

Algorithms/metrics	Silhouette	Calinski–Harabasz	Dunn index	Davies–Bouldin	Sum of norm
Vanilla K-means	0.423 (1.000)	13 932.8 (1.000)	0.352 (1.000)	1.140 (1.000)	4
Bisecting K-means	0.381 (0.622)	12 856.8 (0.719)	0.275 (0.369)	1.224 (0.704)	2.414
Mini-Batch K-means	0.407 (0.856)	13 662.9 (0.928)	0.348 (0.967)	1.144 (0.986)	3.737
Vanilla Gaussian mixture	0.363 (0.459)	12 283.4 (0.569)	0.275 (0.369)	1.238 (0.655)	2.052
Bayesian Gaussian mixture	0.312 (0.000)	10 105.2 (0.000)	0.230 (0)	1.424 (0)	0

Table 6

The difficulty level ordering results on the UoCs generated with the K-means algorithm.

UoC Number (color)	UoC 1 (Green)	UoC 2 (Orange)	UoC 3 (Red)	UoC 4 (Blue)	UoC 5 (Navy)
Feature vector distance	1.108	1.213	1.404	0.953	0.663
Difficulty levels	3	4	5	2	1

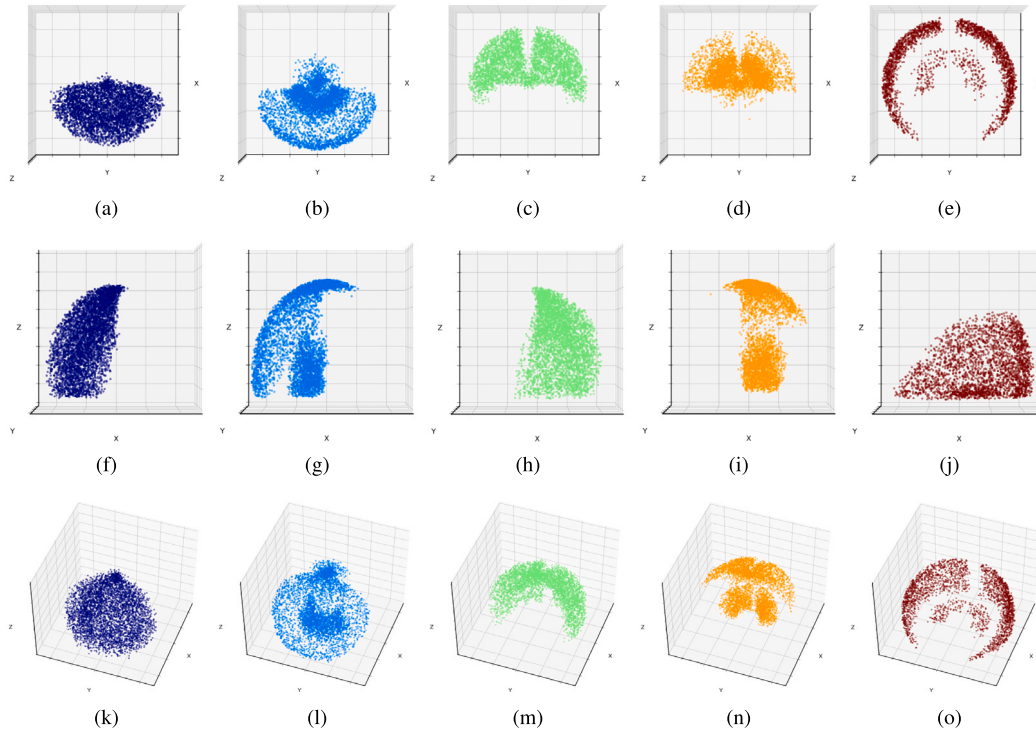


Fig. 7. Visualization results of units of curriculum by different difficulty levels: Each unit of curriculum is generated by the K-means algorithm; the clustered UoC has a symmetrical shape; from left to right, difficulty levels of the units of curriculum are assigned from 1 to 5; (a)–(e): top view, (f)–(j): side view, (k)–(o): isometric view.

Table 7

The verification results on the success rates (%), training times (hour), and training episodes when training the DRL agent without and with the automated curriculum scheme; when applying the automated curriculum scheme, the replay ratio and buffer flushing mechanisms were considered.

Category	Verification results on difficulty levels					Average success rate	Training time	Training episode
	Lv 1	Lv 2	Lv 3	Lv 4	Lv 5			
SAC w/o ACS	94	70	60	40	12	55.2%	37.6 h	38,629
SAC w/ ACS	# 0.0	24	19	17	28	48	27.2%	25.0 h
	# 0.1	52	41	43	40	54	46%	40.9 h
	# 0.2	86	88	72	60	56	72.4%	36.2 h
	# 0.3	89	72	59	56	49	65%	28.7 h
	# 0.4	90	88	78	32	30	63.6%	36.8 h
	# 0.5	93	81	63	43	20	60%	38.5 h
								19,232

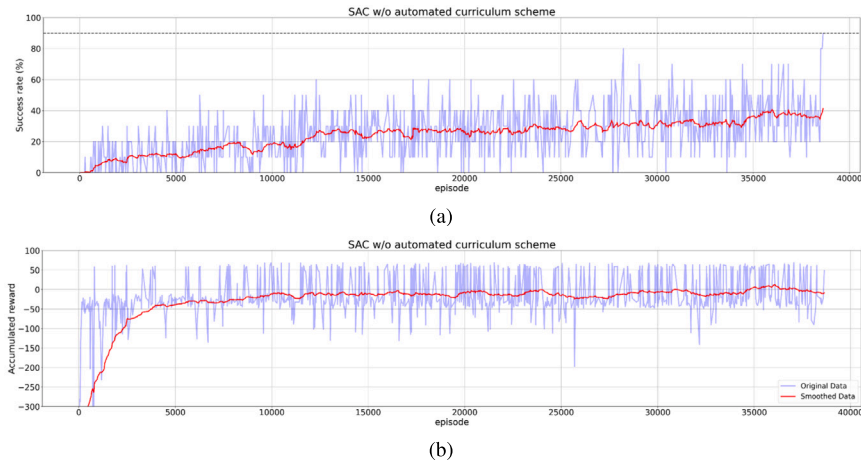


Fig. 8. The results on the success rate and accumulated reward at each episode when training the DRL agent without the designed curriculum scheme.

Table 8

The verification results on success rates (%) of the policy trained without and with the buffer flushing mechanism.

Category	Verification results on difficulty levels					Average success rate
	Lv 1	Lv 2	Lv 3	Lv 4	Lv 5	
w/ buffer flushing #0.2	86	88	70	60	56	72.4%
w/o buffer flushing #0.2	87	83	78	54	40	68.4%

UoCs with low difficulty levels, yet it struggles significantly with the points of the UoCs with higher difficulty levels. This phenomenon occurs because the lower difficulty points tend to provide positive rewards more easily than those of higher difficulty levels. Consequently, these factors lead the agent to settle for local optima where it avoids challenging strategies (i.e., planning motion across the entire spectrum of difficulty levels) and sticks to the existing stable ones (i.e., planning the motion only for the points within lower difficulty levels). In fact, Fig. A.12(a) in Appendix A demonstrates that the entropy of the DRL agent did not fluctuate after the significant decrease, signifying that exploration has not increased. This implies that it is difficult to learn the motion planning task for the entire workspace using traditional DRL algorithms. Deploying such a DRL agent with its trained policy in the real world could lead to frequent failures in achieving the necessary goal points, hindering engineers from obtaining optimal paths for new manufacturing processes in a timely manner.

The SAC w/ ACS #0.0 represents applying the ACS, which does not include a review mechanism, to the baseline SAC algorithm. In other words, the DRL agent does not review the previously learned level's UoCs when learning new ones. The DRL agent achieved higher performance at the most challenging level than the existing baseline SAC approach, but was severely underwhelming at low difficulty levels. This can be called the catastrophic forgetting problem, indicating that the DRL agent forgets how to perform the motion planning task for the previously learned UoCs when transferring to new ones. To address this issue, we applied the devised review mechanism that blends previously learned tasks with the currently learning task during training in accordance with the replay ratios.

When applying the review mechanism to the ACS, the trained policy of the DRL agent achieved positive outcomes across the board, from low to high difficulty levels. Although the success rate marginally falls short of the baseline SAC algorithm at the initial difficulty level, it sustains a certain degree of performance. Furthermore, it outperforms the baseline SAC algorithm from difficulty level 2 onwards. Specifically, the proposed ACS with the replay ratio #0.2 achieves a 44% higher destructive success rate than the baseline algorithm for the most challenging level.

The higher the replay ratio, the more review is done on previously learned levels. When observing the results on the replay ratios from

#0.0 to #0.5 onwards, the success rate at the lowest level, that the agent learned earlier in the curriculum, gradually increase. Conversely, the success rate at the highest level, that the agent learned recently, decrease when the replay ratio increase. These experimental results suggest that the replay ratio generally has a proportional relationship with the success rates of low difficulty levels and an inversely proportional relationship to those of high difficulty levels. Clearly, this shows that the agent trained with the review mechanism does not forget its previous knowledge even as it continuously learns new information. However, an excessive emphasis on review to prevent forgetting previously acquired knowledge can hinder the agent from acquiring new information. Therefore, it is important to mitigate the catastrophic forgetting problem while ensuring performance on the latest learned levels. The key to this is identifying the optimal proportion between review and non-review, meaning the optimal replay ratio.

The replay ratio, yielding the highest average success rate across entire levels, can be defined as the optimal proportion between review and non-review. This is because a high average success rate generally indicates that the agent can consistently plan the motion well from low to high levels. Therefore, the optimal replay ratio in this experiment can be identified as #0.2, indicating that the training target points in each episode are set by the previously trained levels with 20 percent and by the on-going training level for the remaining percent. When the ACS adopts the replay ratio at #0.2, the trained DRL agent surpasses the performance trained with the baseline SAC algorithm from levels 2 to 5, while preserving its capability at the initial level. This experimental result indicates that the optimal replay ratio helps the agent grasp the causality arising from various difficulty levels, ensuring that the knowledge gained at each level complement the others.

As shown in Tables 7, 10, and B.11, the optimal replay ratio for the ACS varies depending on the developed curriculum. When the curriculum consists of 3 or 7 UoCs, higher replay ratios yield better performance. Conversely, when the curriculum consists of 5 difficulty levels, a moderate replay ratio results in better performance. Just as there is an optimal replay ratio for each curriculum, there is also a worst replay ratio. If an agent is trained with a replay ratio that does not align with the curriculum, it may cause significant rollbacks of the policies for previously learned contents, delaying the training

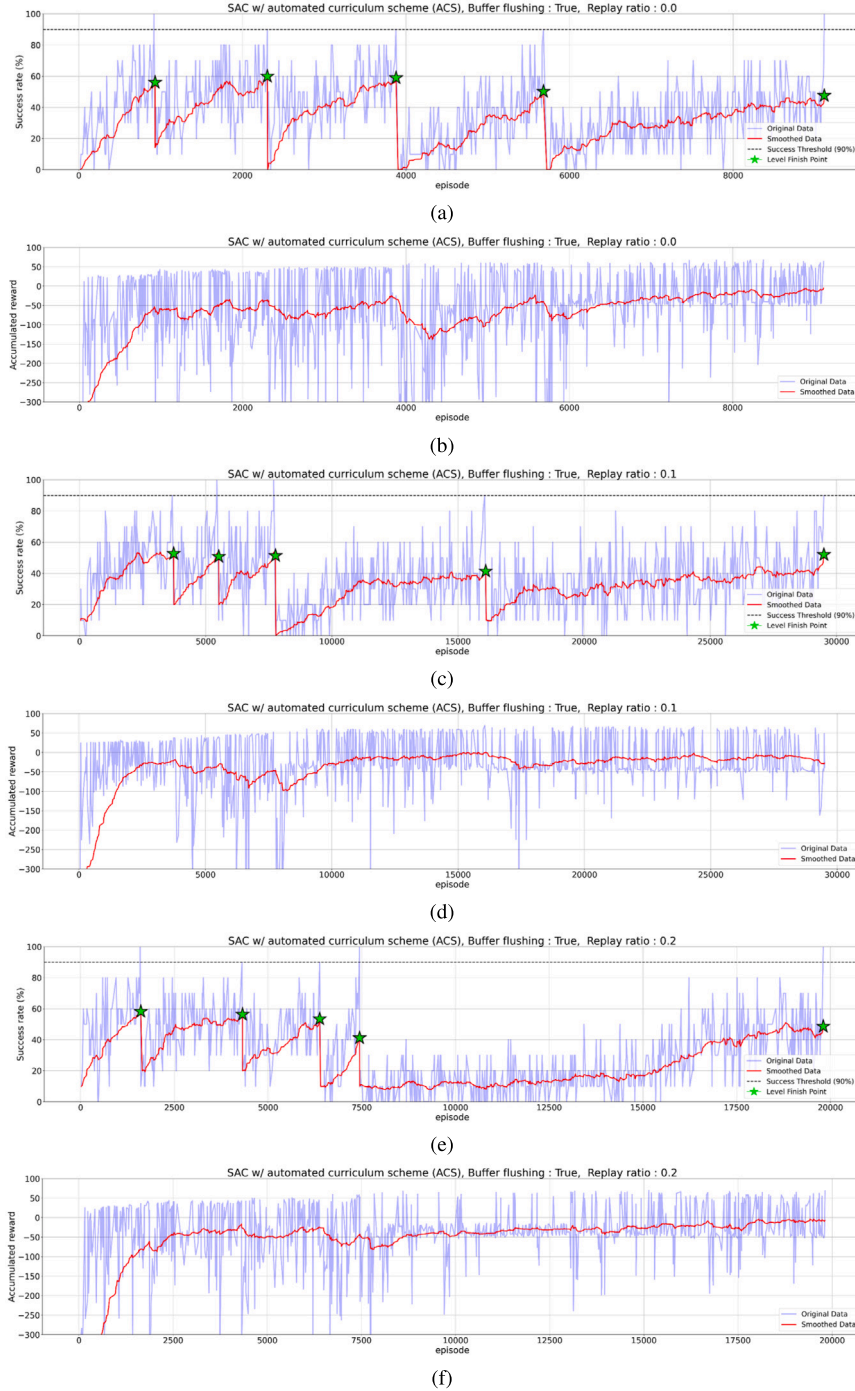


Fig. 9. The results on the success rates and accumulated rewards at each episode when training the DRL agent with the automated curriculum scheme according to different replay ratios; (a–b) #0.0, (c–d) #0.1, (e–f) #0.2.

or severely impacting the agent's overall learning performance. For example, the replay ratio #0.1 results in generally lower learning performance and prolonged learning time for the agent, regardless of the number of UoCs. This is due to the nature of the curriculum learning, which consumes most of the learning time at the highest difficulty level and one level below, as shown in Figs. 9(c), B.13(c) and B.14(c). During this time, the agent remains at a high difficulty level with a low replay ratio, making it prone to forgetting previously learned policies. Therefore, when utilizing this framework, it is expected that excluding excessively low replay ratios will be the most effective way to ensure the agent's learning performance and learning time.

Figs. 9 and 10 show the experimental results of considering the designed ACS. In each Figure, sudden drops in the success rate occur

during learning the curriculum. This is because the difficulty level that the agent is currently learning suddenly changes to the next, signifying that the objective of the currently learning task slightly differs from the previously learned task. After experiencing the level change, the agent continues to train at the changed level and discovers the causality between the previous levels and the current level.

Comparing Fig. 8 with Figs. 9 and 10, it may be seen that the DRL agent without ACS learns more stably in terms of the smoothness of the accumulated reward than the DRL agent with ACS. However, in the case of the DRL agent without ACS, it falls into the easy task preference problem, creating a tendency for the DRL agent to focus excessively on states or actions with already known rewards (i.e., rewards of the easy tasks). This phenomenon is similar to the over-fitting issue caused by

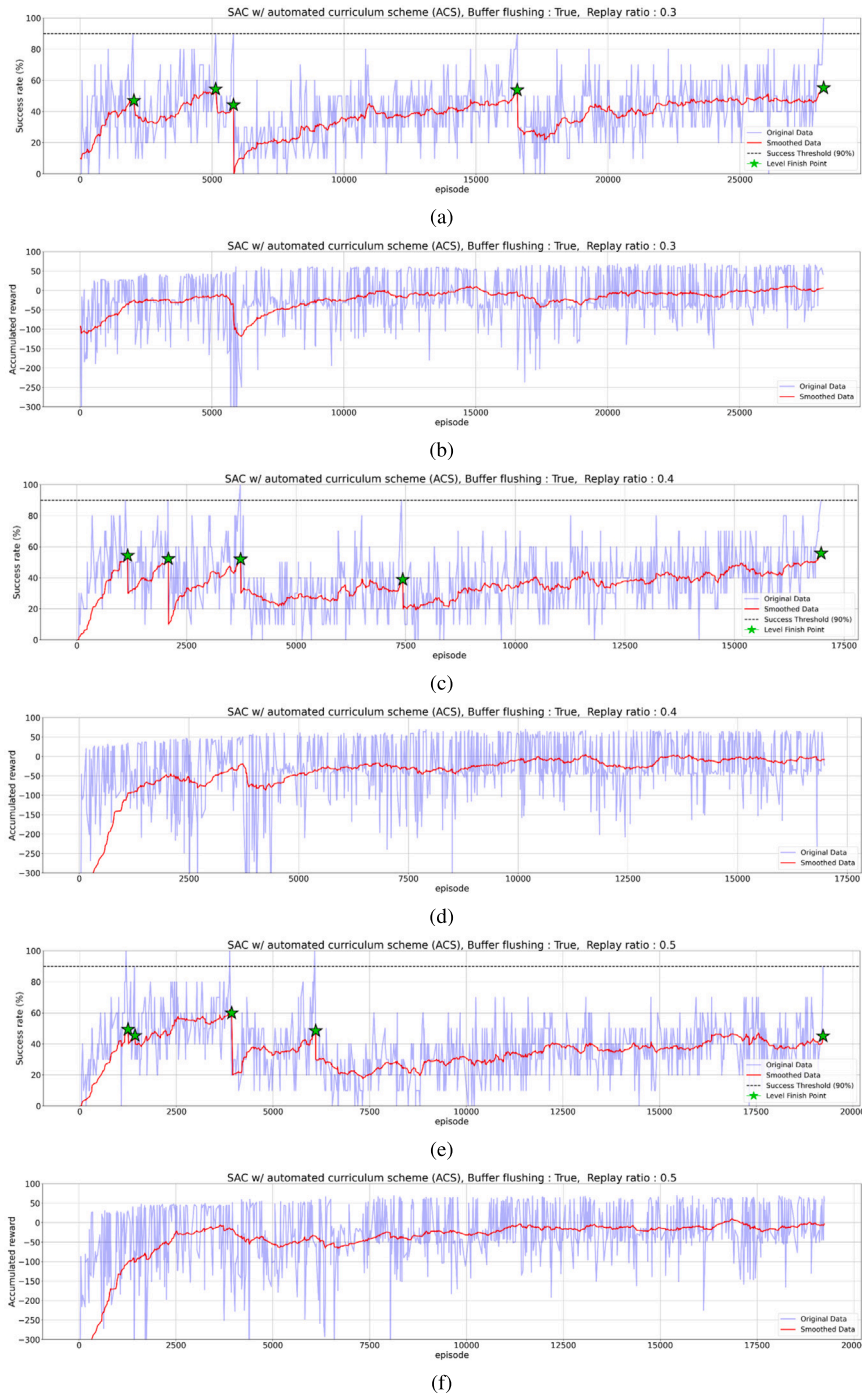


Fig. 10. The results on the success rates and accumulated rewards at each episode when training the DRL agent with the automated curriculum scheme according to different replay ratios; (a–b) #0.3, (c–d) #0.4, (e–f) #0.5.

the class imbalance in the supervised learning. Although the training graph of the DRL agent appears to be stable, as shown in Fig. 8, it has low motion planning accuracy at challenging difficulty levels (Level 4 and Level 5), as shown in Table 7. This indicates that the DRL agent without ACS has stably learned to have an ineffective policy.

On the other hand, the DRL agent with ACS appears to learn more unstably than the agent without ACS. This is because, as mentioned above, when moving to a new difficulty level, the learning task changes and temporarily causes confusion. It takes time for the agent to understand the relationship between the previous difficulty level's tasks and the current difficulty level's task, but it converges quickly after understanding this context. In other words, it seems relatively unstable

during the difficulty level change, but when the agent begins to immerse itself in the curriculum with the new difficulty level, its learning curve gradually stabilizes. This repetitive process from the confusion to the relationship identification results in dramatically higher accuracy in tasks of overall difficulty compared to the DRL agent without ACS. To sum it up, the DRL agent without ACS has stably learned the ineffective policy to plan motions while the DRL agent with ACS defiantly learns the effective policy and its learning curve gradually becomes stable. Fig. 11 shows the success rates of the agents trained with different replay ratios in each difficulty level in detail.

Tables 8 and 9 compare the verification results, both with and without the buffer flushing mechanism in terms of success rates, training

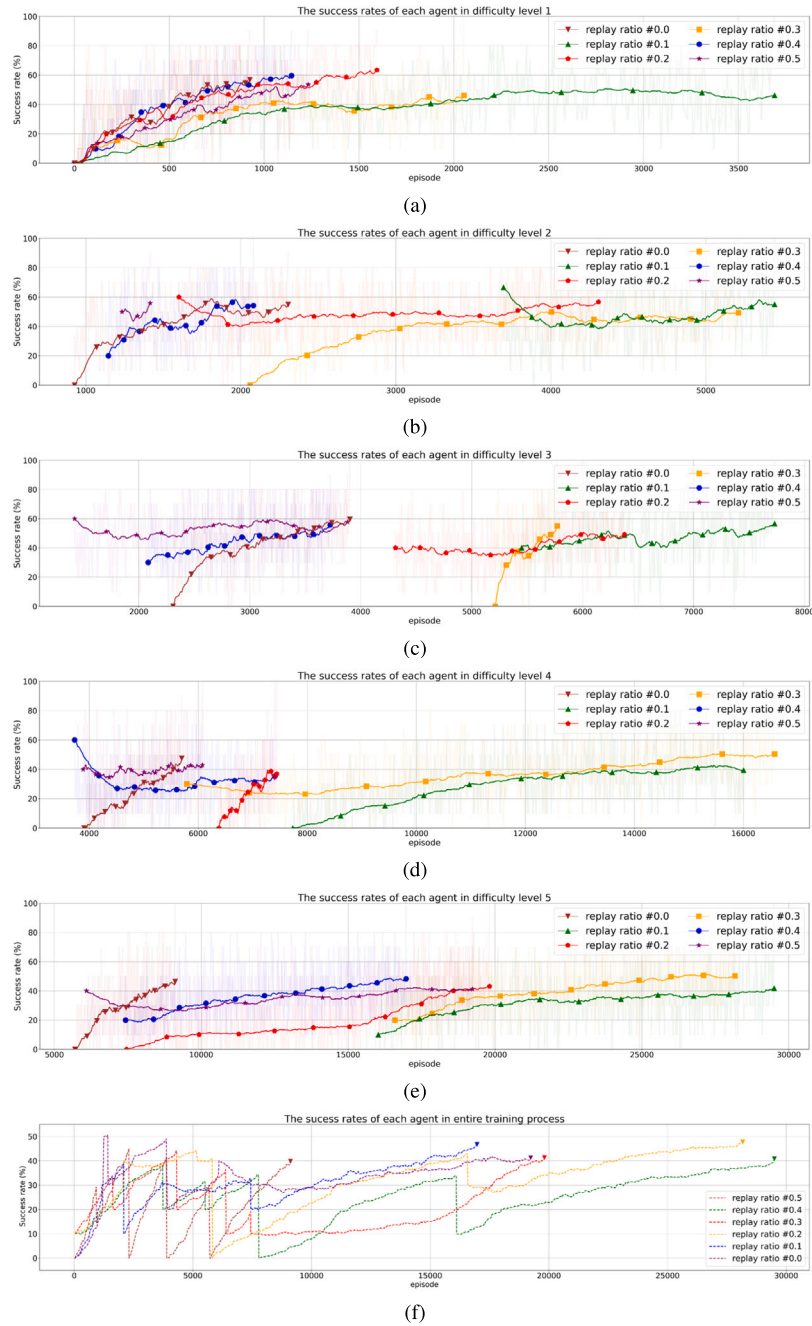


Fig. 11. The results on the success rates of the DRL agents trained with different replay ratios in each difficulty level; (a) difficulty level 1, (b) difficulty level 2, (c) difficulty level 3, (d) difficulty level 4, (e) difficulty level 5, (f) entire difficulty level.

Table 9

The verification results on training times (hour) and episodes of the policy trained without and with the buffer flushing mechanism.

Category	Attribute	Verification results on difficulty levels					Total Result
		Lv 1	Lv 2	Lv 3	Lv 4	Lv 5	
w/ buffer flushing #0.2	Time	5	7.7	5.3	3.6	14.6	36.2
	Episode	1,589	2,716	2,072	1,064	12,370	19,811
w/o buffer flushing #0.2	Time	6.4	13.3	7.1	53.9	63.8	144.5
	Episode	2,451	6,312	3,671	41,872	49,984	104,290

times, and training episodes, when the agent is trained with the replay ratio at #0.2.

As shown in Table 8, the success rates across each difficulty level are generally similar, except for the most challenging level, regardless of

whether the buffer flushing mechanism is applied. This may not impact the low levels significantly, but can greatly affect the accuracy at high levels. This result means that the distribution of data to be learned is important when the agent first encounters a task at the new level. If the

Table 10

The verification results on the success rates (%), training times (hour), and training episodes of the related works and comparison with ours.

Case	Category	Verification results on difficulty levels					Average		Training episode
		Lv 1	Lv 2	Lv 3	Lv 4	Lv 5	success rate	time	
–	SAC w/o ACS (Our baseline)	94	70	60	40	12	55.2%	37.6 h	38,629
case 1	DDPG+direction driven reward (Man et al., 2021)	56	36	30	12	4	27.6%	39.7 h	40,000
case 2	DDPG+joint combination (Ni et al., 2021)	95	64	50	39	34	56.4%	24.5 h	9,453
–	SAC w/ ACS #0.0 (Ours)	24	19	17	28	48	27.2%	25.0 h	9,113
case 3	Our curriculum+SAC+TSCL (abs) (Matiisen et al., 2019)	1	6	14	21	56	19.6%	36.0 h	17,382
–	SAC w/ ACS #0.2 (Ours)	86	88	72	60	56	72.4%	36.2 h	19,811
case 4	Our curriculum+SAC+TSCL (abs/uniform) (Matiisen et al., 2019)	92	71	59	62	31	63.0%	72.0 h	22,823

buffer flushing mechanism is not applied, a large amount of data from previous learned levels will remain in the replay memory of the DRL agent, skewing the learning data distribution towards the UoCs that have already been trained. The data for the currently training level will be less weighted, and the DRL agent cannot prioritize the latest level that needs to be learned at the moment. Consequently, this leads to a relatively low success rate at high levels.

The experimental results in Table 9 show that the proposed buffer flushing mechanism also has a significant impact on training time. In fact, it takes more time and episodes to complete training without the proposed mechanism than with it. These results suggest that the training data was not configured with an ideal replay ratio, complicating the allocation of an appropriate proportion of review and non-review. This is not significantly affected from levels 1 to 3, but the difference in training time becomes extreme at levels 4 and 5. At higher levels, the DRL agent is more sensitive to the distribution of initial training data because it becomes gradually harder to understand the relevant relationships between the previous levels' large amount of accumulated data and the current level's small amount of the data. In other words, the distribution of data during initial training at each levels is significantly important, and the proposed mechanism guides this process.

To compare the proposed G-DRL framework with recent works in terms of applying the DRL algorithm and the curriculum learning to robotic arm motion planning, we implemented their methods and displayed the experimental results, as shown in Table 10 (Man et al., 2021; Ni et al., 2021; Matiisen et al., 2019). Similar to the verification in Table 7, we compared the recent works with our approach in terms of success rates at each difficulty level, average success rate, training time, and training episode.

Before comparing the performance, we briefly describe the four algorithms being compared. In Man et al. (2021), a direction-driven reward function was designed, inspired by the fact that motor cortical cells in the brain are associated with the direction of movement. This function provides a reward to the DRL agent based on the deviation between the actual motion direction and the expected direction of the robotic arm. They developed the DRL agent using the DDPG algorithm and the reward function. In Ni et al. (2021), the DRL agent was trained to reach random target points using three or four DoF. They also utilized the DDPG algorithm and designed three reward functions: basic, done, and extra. In Matiisen et al. (2019), a teacher-student curriculum learning (TSCL) framework was designed where the teacher automatically allocates appropriate tasks to the student, who then tries to learn the allocated task. They proposed a family of teacher algorithms: online, naive, window, and sampling algorithms, which operate as add-on modules. Additionally, a learning curve absolute and uniform sampling mechanisms were designed to prevent the catastrophic forgetting problem. For intuitive explanation, each compared algorithm was named from case 1 to case 4.

To provide fair verification results, we optimized the hyper-parameters for the methods in cases 1 and 2. In cases 3 and 4, the sampling algorithm with the learning curve absolute mechanism was chosen as the teacher due to its highest performance (Matiisen et al., 2019). In cases 1 and 2, the training of the DRL agents was

terminated if the training episode exceeded 40,000 or if the training condition was met. The episode limit of 40,000 was set because the maximum episode count in the experimental results specified in Table 7 is close to 40,000. In cases 3 and 4, the training termination condition was set similarly to Matiisen et al. (2019). The hyper-parameters and the tendency graphs of each training method are shown in Appendix C.

In case 1, the learning condition was not satisfied during 40,000 episodes, so training terminated at episode 40,000. In case 2, the termination condition (i.e., achieving a 90% success rate 5 times in a row) was satisfied, so training terminated at episode 23,566. They consumed 39.7 and 53.4 h of training time, respectively. When comparing the training time of case 2 with our approach (SAC w/ ACS #0.2), it consumed 1.5 times more training time.

As shown in Table 10, the average success rates of cases 1 and 2 are 27.6% and 56.4%, respectively. When examining the verification results at each difficulty level, cases 1 and 2 have a similar trend to the baseline SAC: as the difficulty level of UoCs increased, their success rate tended to decrease. Notably, the agent's performance in case 1 deteriorates more severely than in case 2 as the difficulty level increases. This is because the direction-driven reward system in Man et al. (2021) has the properties of sparse rewards compared to Ni et al. (2021). Generally, sparse rewards tend to induce the easy task preference problem more than dense rewards (Liu et al., 2023b). Although it is less pronounced than in case 1, the agent in case 2, developed through more dense rewards than case 1, also still experiences the problem. This is because the target points for the agent to learn changes randomly in each episode without considering the context of the difficulty level.

Compared to case 2, our approach has higher success rates on both simple and challenging difficulty levels. This indicates that the developed curriculum not only complements the success rate at lower levels through review but also augments the success rate at higher levels by understanding the context of the previously learned difficulty levels and the currently learning difficulty level.

In case 3, the TSCL algorithm was incorporated into our developed curriculum and applied to train the DRL agent. The sampling algorithm, inspired by Thompson sampling, maintains a buffer of the last K rewards for each difficulty level. The teacher algorithm samples recent accumulated rewards from each difficulty level's reward buffer and allocates the task that yielded the highest sampled reward to the student. The teacher assigns the student the task at the difficulty level where the student has the steepest learning curve. In case 4, the uniform sampling mechanism are incorporated to mitigate the forgetting problem observed in case 3.

Cases 3 and 4 are differentiated based on whether the goal of the curriculum learning is viewed as a single task or multiple tasks. Case 3 assumes that the DRL agent can forget about the easy difficulties and only needs to perform well at the most difficult level. On the other hand, in case 4, the goal is not only to plan the motion well at the most challenging difficulty level but also retain performance at easier difficulty levels.

Through the histogram in Fig. C.17 in the Appendix, it can be identified that the teacher in case 3 mainly provided learning content at difficulty levels 1 and 2 in early training stage, levels 3 and 4 in the middle stage, and level 5 at the end stage. The student secured a

high success rate at the highest difficulty level by using the information from levels 1 to 4 as stepping stones. Because the purpose of case 3 is to secure a high success rate at the challenging difficulty level, there is a limitation in that the success rate is not guaranteed at lower difficulty levels. Especially, at levels 1 and 2, which were learned the earlier than others, the success rate is seriously lower than at level 5. This shows a similar tendency to the SAC w/ ACS #0.0, where the replay ratio is set to zero and it experienced the catastrophic forgetting problem. When comparing this with our approach (#0.0), we can observe that it not only takes a longer time to learn, but also results in relatively low average accuracy. At the challenging difficulty level, it achieves the same accuracy with SAC w/ ACS #0.2 which has the highest performance in our approach. However, case 3 achieved significantly lower accuracy at low difficulty levels due to the catastrophic forgetting problem, despite having a similar learning time to SAC w/ ACS #0.2.

On the other hand, the average success rate of case 4 achieves 63% with 72 training hours. Compared to case 3, the uniform sampling mechanism in case 4 significantly enhance the success rate, excluding the most challenging difficulty level. These experimental results indicate that mixing the previously learned difficulty levels while learning the current difficulty level is important to secure consistent performance from low levels to high levels.

When comparing case 4 with SAC w/ ACS #0.2, it overall underperformed except at difficulty level 1. Case 4's reviewing mechanism was conceptually similar to the G-DRL framework except for the details. The difference is that our approach temporarily fixed the difficulty level and enhanced the learning sequence once the training is finished. Our approach made the DRL agent learn towards a fixed learning objective and changed it little by little whenever the training was finished on the UoC, allowing the agent to quickly adapt without being very confused about the changed learning objective. On the other hand, case 4's training mechanism assigned the student a difficulty level at which the student initially tended to learn well, without considering whether it was easy or difficult. In addition, case 4 assigned the difficulty level to be learned for the next episode based on the sampling results, regardless of the difficulty level being learned in the current episode. These detailed differences nearly doubled the learning time and caused an accuracy difference of nearly 10%.

Therefore, it can be said that the key effective learning is twofold: sequentially progressing through difficulty levels from easy to challenging, and maintaining a relationship between the previously learned and the currently learning difficulty levels. These experimental results confirmed that the proposed approach outperformed not only the baseline algorithm but also recent works. Furthermore, the experimental results showing that the performance of the DRL agent in the baseline, case 1, and case 2 decreases as the difficulty level of the UoC increases, and that TSCL, the automatic curriculum assignment methodology, assigned difficulty levels 1 through 5 sequentially, indicated that our curriculum generation methodology was well-designed, including clustering data points and assigning difficulty levels sequentially based on features.

In summary, these experimental results indicate that the two designed mechanisms are very effective in supporting the ACS-based DRL agent training. These mechanisms have overall advantages in terms of the success rates and training time over not applying them. The DRL agent trained with ACS successfully learns the motion planning task for the entire range of the robot's reach, which has large-scale search spaces, and the verification results show that it outperforms the traditional DRL framework (i.e., SAC) and recent works. The experimental result in an environment with obstacles and experimental result on the sim2real test are shown in [Appendix D](#) and [E](#), respectively.

7. Discussion - applicability to other domains

The core workflow of the proposed framework generates the feature data from the existing knowledge-based surrogate feature generation model representing the difficulty by proxy, constructs the units of

curriculum through clustering, and builds the relative difficulty level sequence tailored to the characteristics of the features. This workflow can be extensively applied not only to robotic arms but also to autonomous vehicles (AVs), unmanned aerial vehicles (UAVs), and automated guided vehicles (AGVs). In this section, we discuss the applicability of the proposed workflow to other domains.

The proposed framework can be used to train AVs to adaptively respond to various weather conditions, ranging from clear skies to severe weather. It utilizes a weather analysis model and extracts the feature data such as the expected light levels, road friction, and cloud cover from each scenario. Then, it generates the units of curriculum and builds the level sequence based on the generated features. The generated curriculum will be able to teach the motion planner of the AVs to operate adaptively in changing environments.

The proposed framework can be utilized to train AGVs to effectively transport logistics in congested traffic. It utilizes traffic flow analysis and population density expectation, environment models to extract the features related to traffic volume, obstacles, and logistics priority in each scenario. The extracted features can be used to construct the curriculum and build the level sequence based on the generated features. The motion planner of the AGVs can learn the scenarios according to the difficulty level of the curriculum.

The proposed framework may also be able to utilized to train drones to autonomously search dangerous areas that are difficult for humans to access. The framework employs the environment and battery usage expectation model so that it extracts the features such as the vegetation density, navigation paths, and expected available battery. Then, it generates the units of curriculum and builds the level sequence for each scenario that the agent is needed to learn. As expected, the drone trained with the curriculum may have the ability to search danger areas regardless the geographical property.

8. Conclusion

This paper focuses on solving the easy task preference problem that occurs during training the DRL agent to plan the motion for the entire workspace of a robotic arm. To this end, the proposed framework extracts features that can represents the difficulties for reaching target points, generates the curriculum which can reflect the causality stemming from the heterogeneous difficulty levels. The proposed framework sequentially trains the DRL agent according to the developed curriculum.

To verify the effectiveness of the proposed framework, experiments were conducted on the generation of curriculum units, the assignment of relative difficulty levels, and the training of the DRL agent based on the curriculum. The experimental results demonstrate that the proposed framework can generate optimal curriculum units for training the motion planning task across the entire workspace and determine their relative difficulty levels. Moreover, when the DRL agent was trained using the developed curriculum, the result overwhelms the performance of the baseline DRL framework, especially at high difficulty levels. These results indicate that by adding the context of reaching difficulty, which was not considered in existing methodologies, the DRL agent can stably learn the motion planning task for target points across the entire workspace.

Generally, traditional physics-based methods cannot provide near-optimal solutions in a timely manner due to computational complexity, while existing data-driven methods significantly under-perform in training the motion planning task for the entire workspace, which are the large-scale search spaces. In contrast, the proposed framework proves to be more effective than traditional physics-based and data-driven methodologies when operators in a factory develop new manufacturing processes. In other words, when operators perform teaching for industrial robots to reach specific target positions, the proposed framework can provide optimal plans more promptly than existing methods. This is particularly advantageous when operators request motion planning for target points that are difficult to reach.

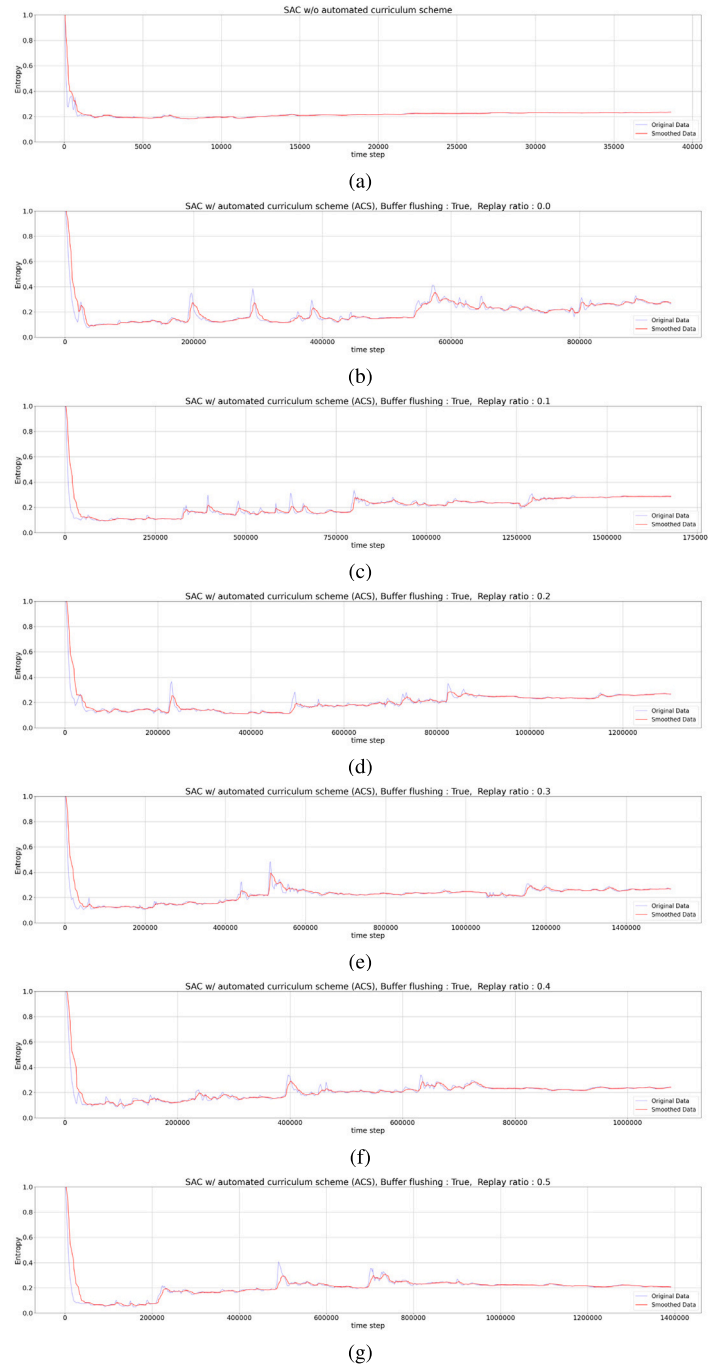


Fig. A.12. The results on the entropies at each episode when training the DRL agents with the number of UoCs 5; (a) without the automated curriculum scheme, (b–g) with the automated curriculum scheme.

CRedit authorship contribution statement

Deun-Sol Cho: Writing – review & editing, Writing – original draft, Project administration, Methodology, Formal analysis, Data curation, Conceptualization. **Jae-Min Cho:** Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation. **Won-Tae Kim:** Writing – review & editing, Supervision, Resources, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partly supported by the ICT R&D program of MSIT/IITP, Republic of Korea [IITP-2022-0-00866, Development of manufacturing digital twin framework technology that supports high-fidelity and distributed simulation for large-scalability] and [IITP-2022-0-00438, Development of core technology for the operation and prediction of intelligent digital twin federation] in 2024.

Appendix A. Entropy of the DRL agents when training

See Fig. A.12.

Table B.11

The verification results on the success rates (%), training times (hour), and terminated episodes when training the DRL agent without and with the automated curriculum scheme; when applying the automated curriculum scheme, the replay ratio and buffer flushing mechanisms were considered; in the experiments, the policy of the baseline SAC is the same as the number of UoCs is 5; this experiment is conducted when the number of UoCs is 3.

Category		Verification results			Average success rate	Training time	Spent episode
		Lv 1	Lv 2	Lv 3			
SAC w/o ACS					55.2%	37.6 h	38,629
SAC w/ ACS	# 0.0	21	28	34	27.6%	20.4 h	6,793
	# 0.1	42	53	53	49.6%	24.7 h	13,579
	# 0.2	72	65	42	59.6%	22.1 h	10,327
	# 0.3	82	63	26	57%	26.0 h	13,455
	# 0.4	79	66	41	62%	24.1 h	13,190
	# 0.5	84	70	39	64.3%	24.7 h	10,615

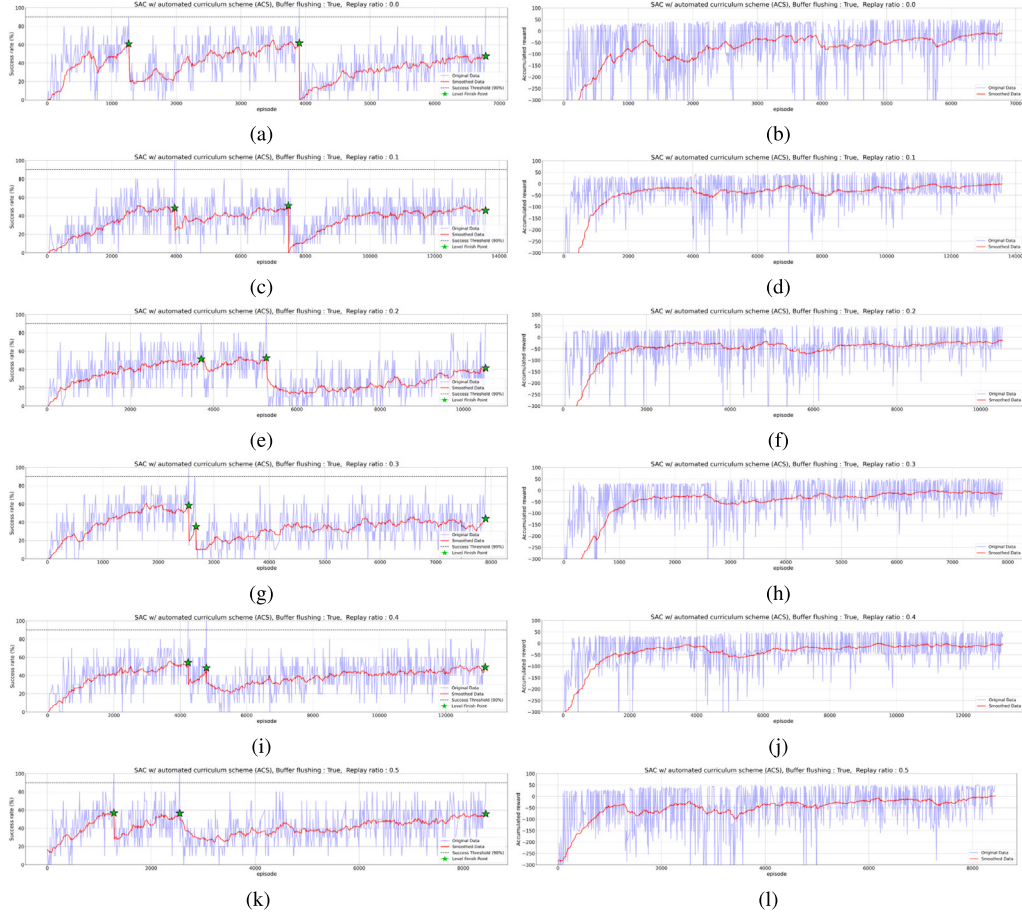


Fig. B.13. The results on the success rates and accumulated rewards at each episode when training the DRL agent with the automated curriculum scheme according to different replay ratios; (a–b) #0.0, (c–d) #0.1, (e–f) #0.2, (g–h) #0.3, (i–j) #0.4, (k–l) #0.5.

Appendix B. Verification results when other number of UoCs

See Figs. B.13, B.14, Tables B.11 and B.12.

Appendix C. Hyper-parameters and training graphs of the recent works

See Figs. C.15–C.17, Tables C.13 and C.14.

Appendix D. Experimental results in an environment with obstacles

To verify the scalability of the proposed framework for obstacle-containing environments and various robotic arm platforms, we constructed a virtual simulation environment that replicates a real-world

setting with obstacles, as shown in Fig. D.18(b). The robotic arm mounted on the mobile base is the Doosan H2017 model. Using the gazebo simulator, we accurately simulate the dynamics of the robotic arm and detect collisions between the robotic arm and other objects.

For the experiment, we extracted 2,000 target points (in contrast to 20,000 target points in the previous experiment detailed in the main text) that do not collide with the obstacles as shown in Fig. D.19. We generated the UoCs based on the curriculum unit generator and constructed the curriculum using the learning sequence builder. The curriculum manager then applies the constructed curriculum when the DRL agent learns the motion planning task. Figs. D.20–D.22 show the results on the success rates when training the DRL agent.

As shown in Table D.15, the DRL agent trained with the proposed framework achieves an average success rate of 55.2%, with a training time of 34.3, and 7,110 training episodes. In contrast, the methods

Table B.12

The verification results on the success rates (%), training times (hour), and terminated episodes when training the DRL agent without and with the automated curriculum scheme; When applying the automated curriculum scheme, the replay ratio and buffer flushing mechanisms were considered; in the experiments, the policy of the baseline SAC is the same as the number of UoCs is 5; this experiment is conducted when the number of UoCs is 7.

Category	Verification results on difficulty levels								Average	Training	Spent
		Lv 1	Lv 2	Lv 3	Lv 4	Lv 5	Lv 6	Lv 7	success rate	time	episode
SAC w/o ACS									55.2%	37.6 h	38,629
SAC w/ ACS	# 0.0	29	28	4	15	38	9	56	25.5%	31.9 h	19,116
	# 0.1	53	64	36	43	67	18	51	47.4%	40.5 h	29,115
	# 0.2	84	77	68	54	66	29	50	61.1%	46.2 h	26,169
	# 0.3	80	54	79	68	54	49	48	61.7%	42.5 h	15,935
	# 0.4	79	76	67	51	77	16	46	58.8%	34.6 h	15,743
	# 0.5	90	92	80	74	87	27	43	70.4%	35.2 h	13,756

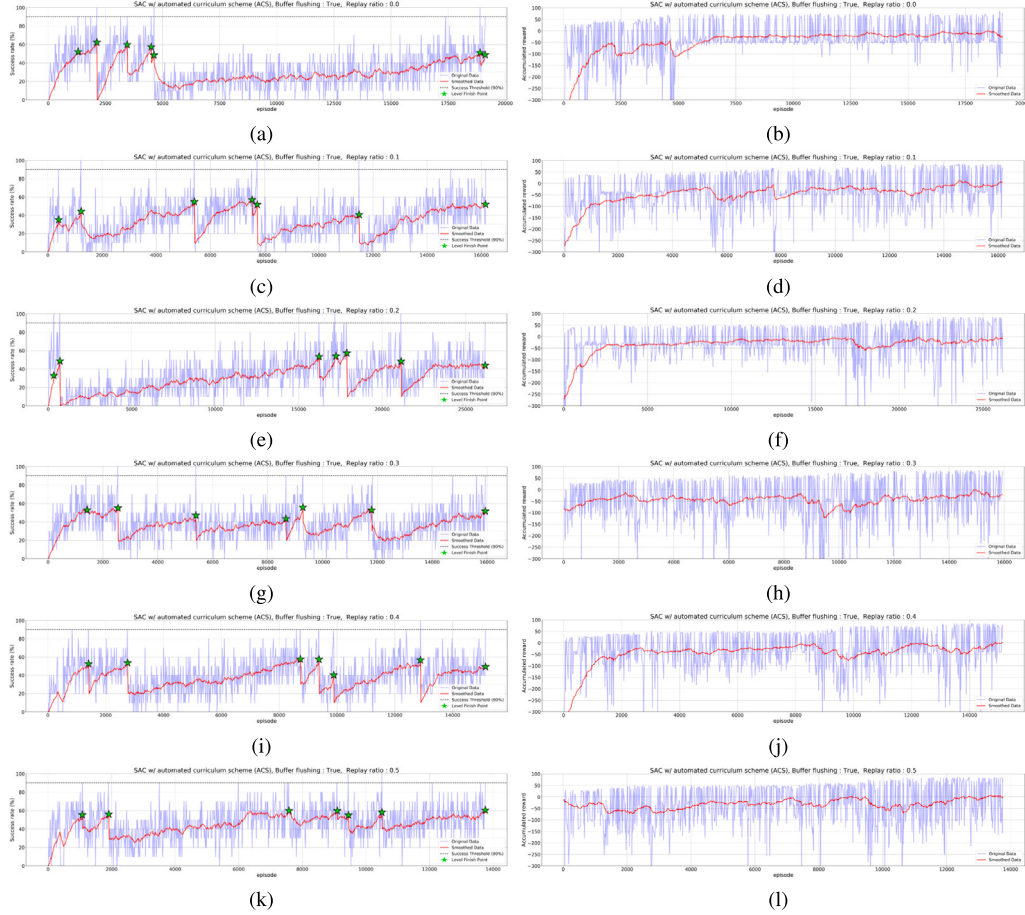


Fig. B.14. The results on the success rates and accumulated rewards at each episode when training the DRL agent with the automated curriculum scheme according to different replay ratios; (a–b) #0.0, (c–d) #0.1, (e–f) #0.2, (g–h) #0.3, (i–j) #0.4, (k–l) #0.5.

proposed by Man et al. (2021) and Ni et al. (2021) significantly underperform than our approach in terms of the training accuracy. This result suggests that the context that each target point has different motion planning difficulty has a significant impact on the agent's learning, even in environments with obstacles. Additionally, the performance difference becomes more pronounced when the ACS is applied compared to when it is not.

Fig. D.23 illustrates the trajectory generated by the DRL agent trained using the ACS. The robotic arm starts in an upright position, and the target point corresponds to the level 5, which presents the highest level of motion planning difficulty. Despite the presence of obstacles in front of and below the robotic arm, the DRL agent successfully generates an optimal path that avoids these obstacles. These experimental results demonstrate that the proposed framework can effectively train

DRL agents even in obstacle-laden environment, and that the proposed framework can be applied to various robotic arm hardware platforms.

Appendix E. Experimental results on the Sim2real test

To demonstrate the feasibility of the proposed framework, we identically constructed the real world environment and the virtual world environment in the gazebo simulator as shown in Fig. D.18(a) and (b). Each environment includes the robotic arm (Doosan H2017), grippers, the gripper stand, and the mobile base.

As illustrated in Fig. E.24, we developed two controllers that control the real-world robot and virtual-world robot according to a generated motion plan. First, the DRL agent within the framework generates the motion plan from the starting point to the target point. Then,

Table C.13

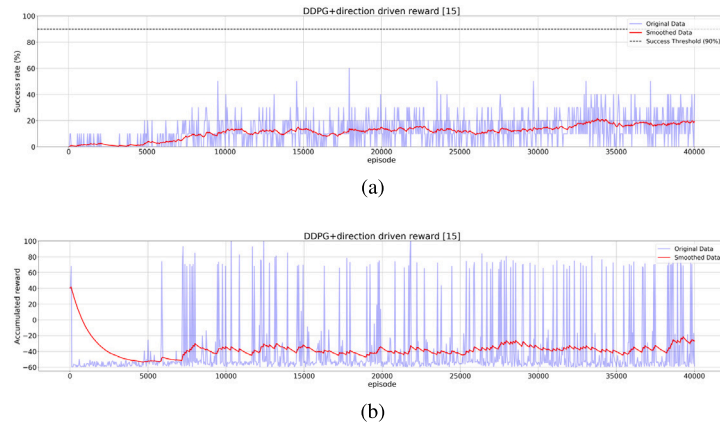
The hyper-parameters for training the DRL agents using recent works (Man et al., 2021; Ni et al., 2021).

Hyper-parameters	Man et al. (2021)	Ni et al. (2021)
DDPG algorithm		
Learning rate	0.00015	0.00015
Discount factor	0.99	0.99
Replay buffer size	1,000,000	1,000,000
Number of hidden units	128*64*32	64*64
Number of minibatch sample	1024	1024
Optimizer	Adam	Adam
Activation function	Linear	Linear
C (Negative episodic reward)	-10	-10
S (Positive episodic reward)	50	50
δ (Control factor)	50	50
μ	0	0
τ	0.005	0.005
θ	0.1	0.1
σ	0.5	0.5
σ_{min}	0.05	0.05
σ_{decay}	0.99	0.99

Table C.14

The hyper-parameters for training the DRL agents using the TSCL (Matiisen et al., 2019).

Hyper-parameters	Matiisen et al. (2019)
SAC algorithm (student)	
Learning rate	0.00015
Discount factor	0.99
Replay buffer size	20,000
Number of hidden units	64*64
Number of minibatch sample	4096
Optimizer	Adam
Activation function	Linear
C (Negative episodic reward)	-10
S (Positive episodic reward)	50
δ (Control factor)	50
K (sampling buffer)	10
Sample algorithm (teacher)	
Learning curve absolute mechanism	True
uniform sampling mechanism	True/False
ϵ	0.1

**Fig. C.15.** The DRL agent's results on the success rate and accumulated reward at each episode when using the DDPG algorithm and direction driven system (Man et al., 2021).**Table D.15**

The verification results on the success rates (%), training times (hour), and training episodes of the related works and comparison with ours.

Category	Verification results on difficulty levels					Average	Training	Training
	Lv 1	Lv 2	Lv 3	Lv 4	Lv 5	success rate	time	episode
SAC w/ ACS #0.2 (ours)	66	62	71	58	78	67%	34.3 h	7,110
DDPG+direction driven reward (Man et al., 2021)	18	15	7	2	2	8.8%	29.5 h	20,000
DDPG+joint combination (Ni et al., 2021)	87	78	41	24	6	47.2%	20.0 h	7,552

the framework provides the generated plan to the controllers. Upon receiving the motion plan, the virtual robot controller activates the necessary ROS nodes, enabling the robotic arm in the Gazebo simulator

to execute the motion plan. On the other hand, the real robot controller generates a script according to the motion plan and controls the robotic arm in the real world.

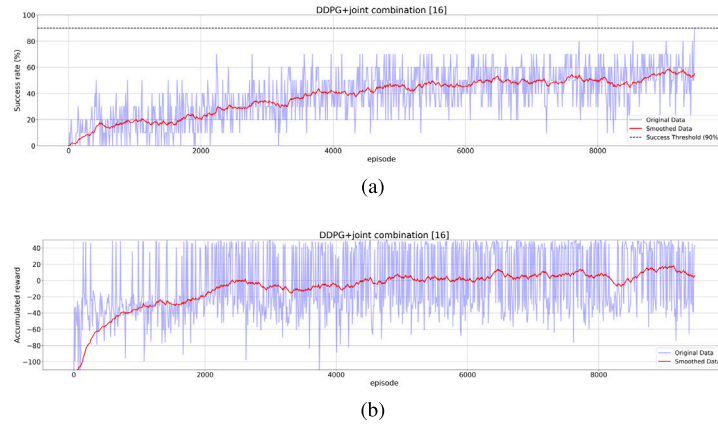


Fig. C.16. The DRL agent's results on the success rate and accumulated reward at each episode when using the DDPG algorithm and joint combination approach (Ni et al., 2021).

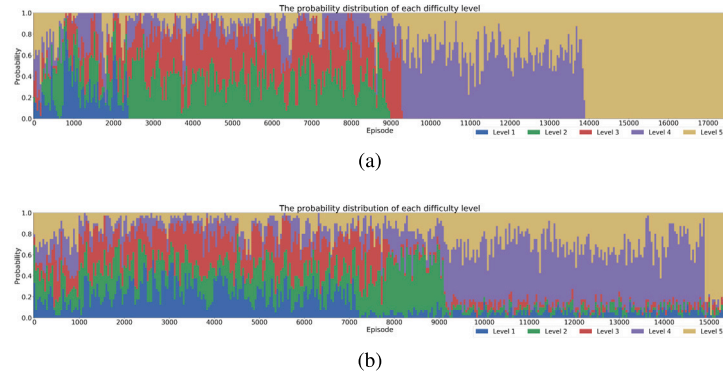


Fig. C.17. The histograms on the distribution of probability that the teacher allocates the difficulty level to the student in each episode; (a) with the learning curve absolute mechanism only, (b) with the learning curve absolute and uniform sampling mechanisms.

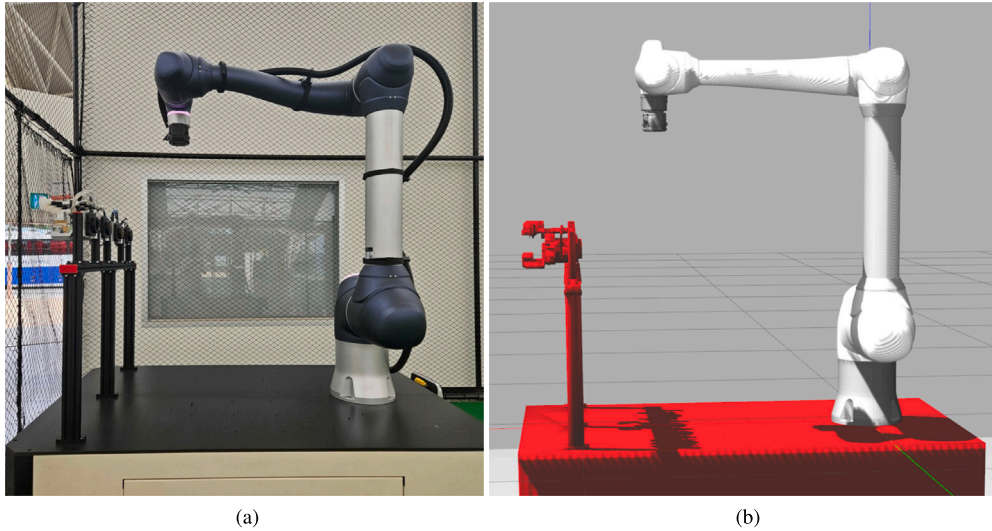


Fig. D.18. The experimental environments (a) the robotic arm constructed in real environment, (b) the robotic arm constructed in virtual simulation environment (gazebo); The objects colored red (grippers, gripper stand, and mobile base) were set as obstacles.

Fig. E.25 presents the trajectories of the real and virtual robotic arms controlled by the controllers. As shown in Fig. E.25(a) and (b), the trajectories in real world and virtual simulation world exhibit visually similar pattern. To verify this numerically, we utilize the R^2 score which assesses the alignment between the two datasets. The R^2 score ranges from 0 to 1, with a higher value indicating a better fit between the

simulation and real-world data. The R^2 scores of the trajectories in Fig. E.25(a) and (b) were 1.0 and 0.999, respectively. This experimental result demonstrates that the Gazebo simulator, employed in the proposed framework for training the DRL agent on the motion planning task is reliable, and the generated motion plan is feasible for controlling real-world robot arms.

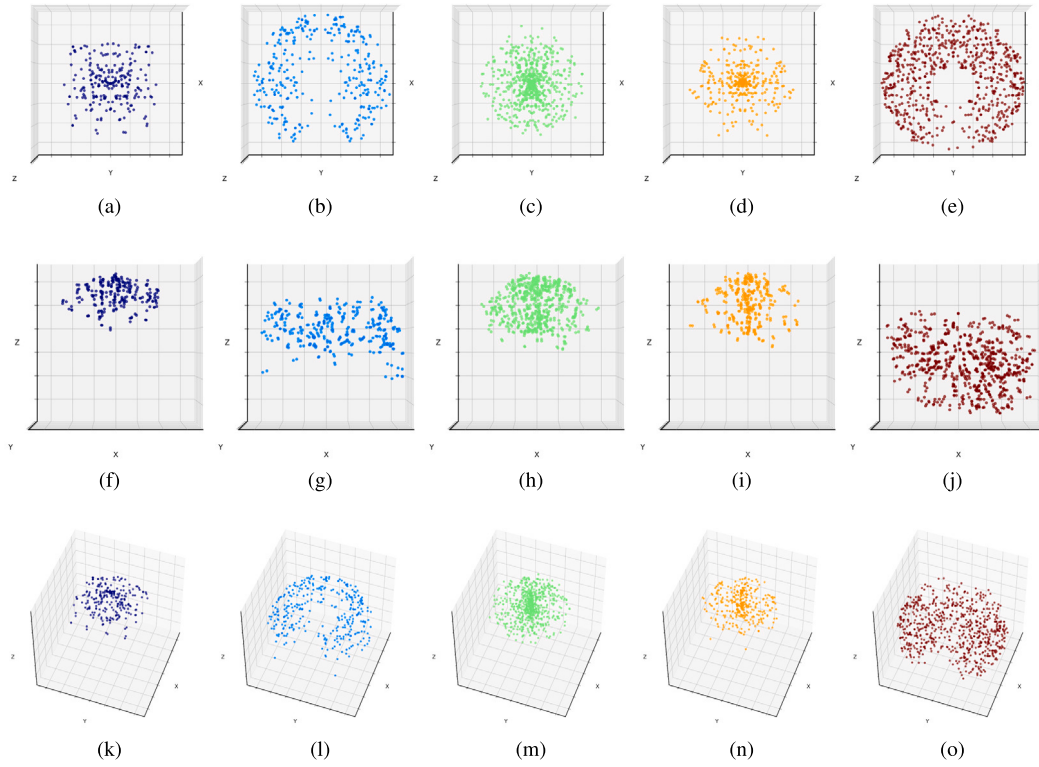


Fig. D.19. Visualization results of units of curriculum by different difficulty levels: Each unit of curriculum is generated by the K-means algorithm; the clustered UoC has a symmetrical shape; from left to right, difficulty levels of the units of curriculum are assigned from 1 to 5; (a)–(e): top view, (f)–(j): side view, (k)–(o): isometric view.

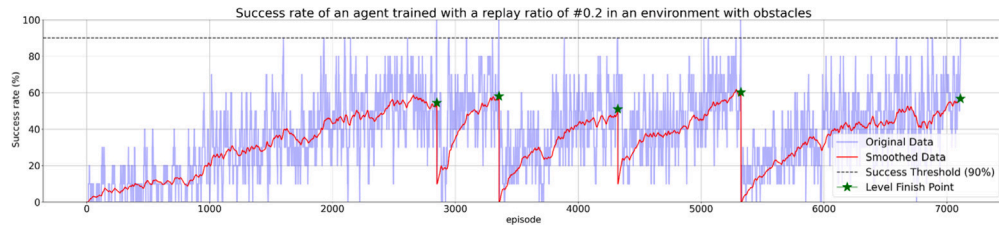


Fig. D.20. The results on the success rates at each episode when training the DRL agent in an environment with obstacles according to replay ratio #0.2.

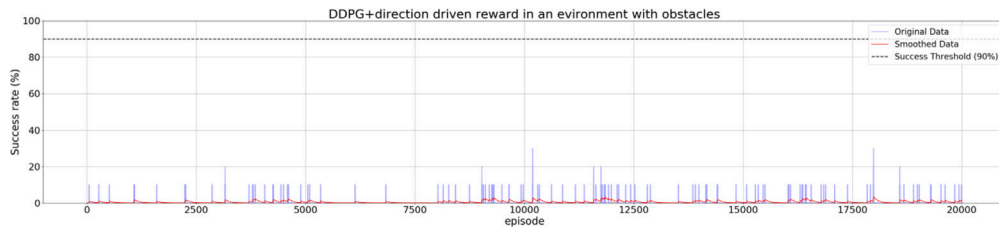


Fig. D.21. The results on the success rates at each episode when training the DRL agent using the DDPG algorithm and direction driven system (Man et al., 2021).

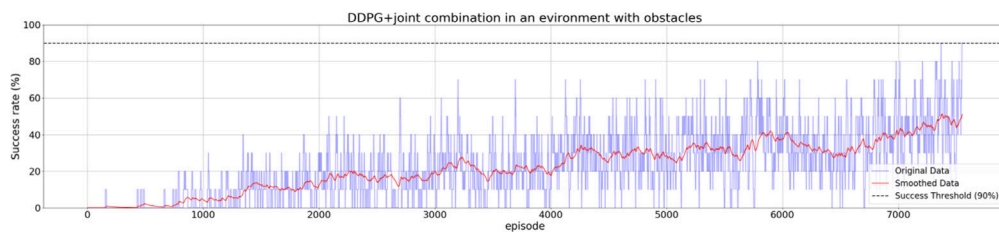


Fig. D.22. The results on the success rates at each episode when training the DRL agent using the DDPG algorithm and joint combination approach (Ni et al., 2021).

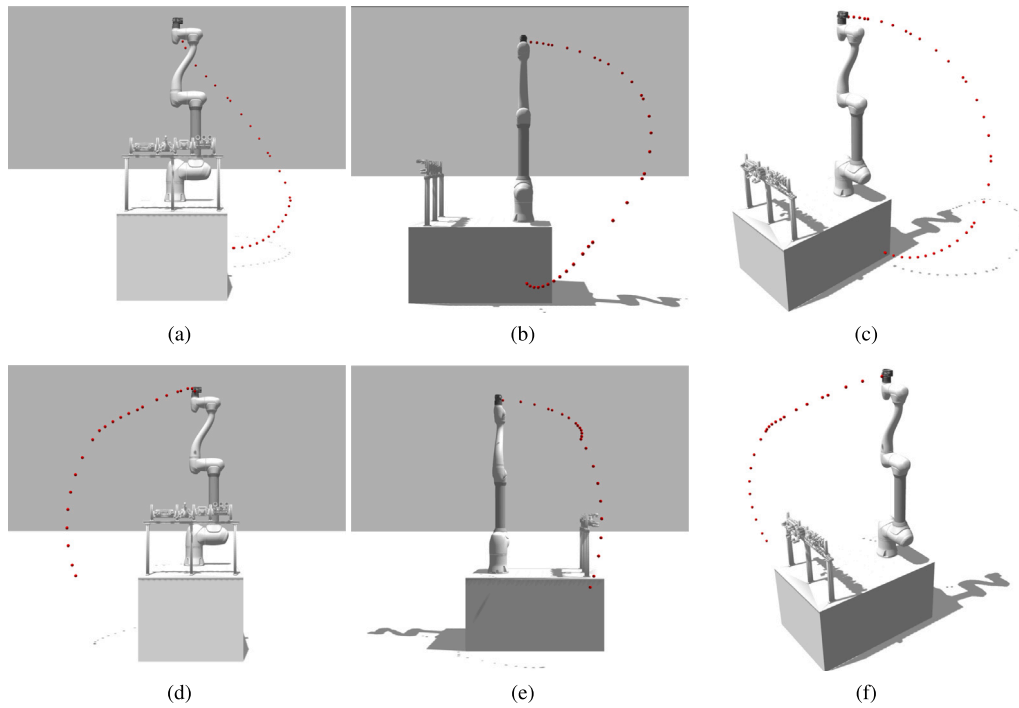


Fig. D.23. The trajectory generated by the DRL agent trained using the automated curriculum scheme.

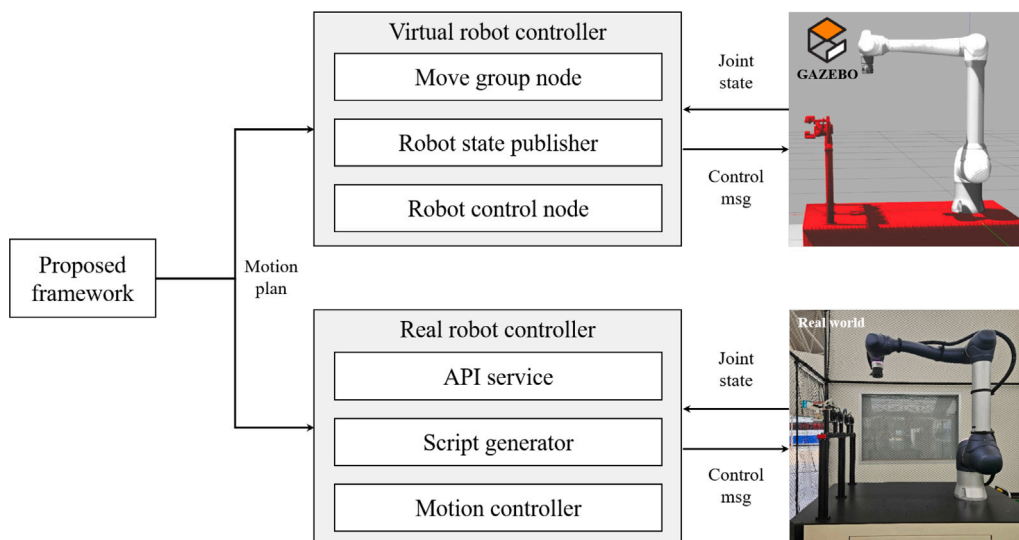


Fig. E.24. The comparison between real and virtual robotic arm's trajectory; Each robotic arm was controlled by the motion plan generated by the DRL agent from Appendix D; (a) the result mapped with the motion plan from Fig. D.23(a–c), (b) the result mapped with the motion plan from Fig. D.23(d–f).

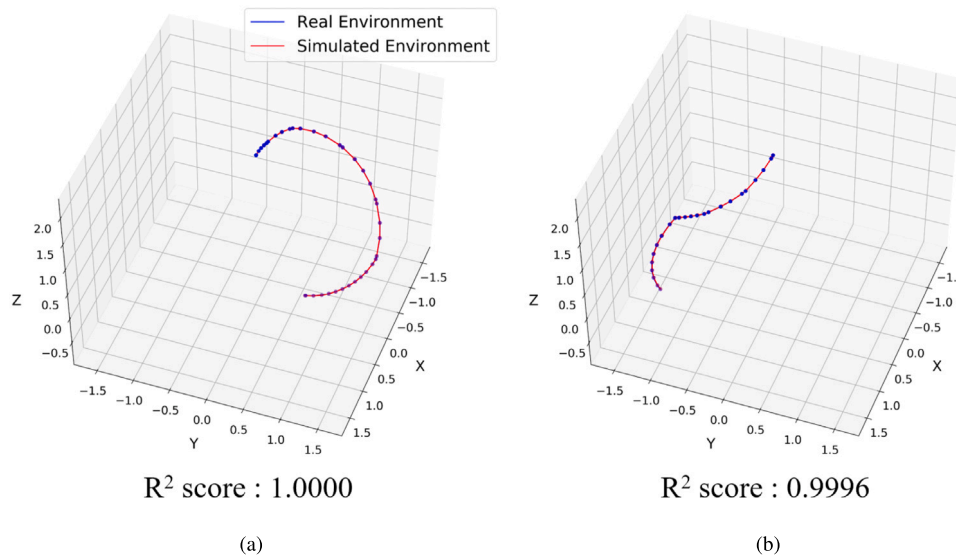


Fig. E.25. The comparison between real and virtual robotic arm's trajectory; Each robotic arm was controlled by the motion plan generated by the DRL agent from Appendix D; (a) the result mapped with the motion plan from Fig. D.23(a-c), (b) the result mapped with the motion plan from Fig. D.23(d-e).

Data availability

Data will be made available on request.

References

- Agüero, C.E., Koenig, N., Chen, I., Boyer, H., Peters, S., Hsu, J., Gerkey, B., Paepcke, S., Rivero, J.L., Manzo, J., et al., 2015. Inside the virtual robotics challenge: Simulating real-time robotic disaster response. *IEEE Trans. Autom. Sci. Eng.* 12 (2), 494–506.
- Altman, E., 2021. *Constrained Markov Decision Processes*. Routledge.
- Annaswamy, A.M., 2023. Adaptive control and intersections with reinforcement learning. *Ann. Rev. Control Robot. Auton. Syst.* 6, 65–93.
- Barjuei, E.S., Capitanelli, A., Bertolucci, R., Courteille, E., Mastrogianni, F., Maratea, M., 2024. Digital workflow for printability and prefabrication checking in robotic construction 3D printing based on artificial intelligence planning. *Eng. Appl. Artif. Intell.* 133, 108254.
- Basile, G., Lui, D.G., Petrillo, A., Santini, S., 2024. Deep deterministic policy gradient virtual coupling control for the coordination and manoeuvring of heterogeneous uncertain nonlinear high-speed trains. *Eng. Appl. Artif. Intell.* 133, 108120.
- Behere, S., 2010. *A Generic Framework for Robot Motion Planning and Control*. Royal Institute of Technology, Stockholm, Sweden.
- Coleman, D., Sucan, I., Chitta, S., Correll, N., 2014. Reducing the barrier to entry of complex robotic software: a moveit! case study. *arXiv preprint arXiv:1404.3785*.
- Ekrem, Ö., Aksoy, B., 2023. Trajectory planning for a 6-axis robotic arm with particle swarm optimization algorithm. *Eng. Appl. Artif. Intell.* 122, 106099.
- Guan, H., 2020. Analysis on deep reinforcement learning in industrial robotic arm. In: 2020 International Conference on Intelligent Computing and Human-Computer Interaction. ICHCI, IEEE, pp. 426–430.
- Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *International Conference on Machine Learning*. PMLR, pp. 1861–1870.
- Kang, Y., Kim, D., Kim, K., 2019. URDF generator for manipulator robot. In: 2019 Third IEEE International Conference on Robotic Computing. IRC, IEEE, pp. 483–487.
- Kanitscheider, I., Huizinga, J., Farhi, D., Guss, W.H., Houghton, B., Sampedro, R., Zhokhov, P., Baker, B., Ecoffet, A., Tang, J., et al., 2021. Multi-task curriculum learning in a complex, visual, hard-exploration domain: Minecraft. *arXiv preprint arXiv:2106.14876*.
- Kerzel, M., Mohammadi, H.B., Zamani, M.A., Wermter, S., 2018. Accelerating deep continuous reinforcement learning through task simplification. In: 2018 International Joint Conference on Neural Networks. IJCNN, IEEE, pp. 1–6.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al., 2017. Overcoming catastrophic forgetting in neural networks. *Proc. Natl. Acad. Sci.* 114 (13), 3521–3526.
- Li, Y., 2017. Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274*.
- Li, Z., Ma, H., Ding, Y., Wang, C., Jin, Y., 2020. Motion planning of six-dof arm robot based on improved DDPG algorithm. In: 2020 39th Chinese Control Conference. CCC, IEEE, pp. 3954–3959.
- Liu, Z., Li, R., Zhao, L., Xia, Y., Qin, Z., Zhu, K., 2023a. Automatic joint motion planning of 9-DOF robot based on redundancy optimization for wheel hub polishing. *Robot. Comput.-Integr. Manuf.* 81, 102500.
- Liu, S., Liu, P., 2021. A review of motion planning algorithms for robotic arm systems. In: *RITA 2020: Proceedings of the 8th International Conference on Robot Intelligence Technology and Applications*. Springer, pp. 56–66.
- Liu, W., Niu, H., Mahyuddin, M.N., Herrmann, G., Carrasco, J., 2021. A model-free deep reinforcement learning approach for robotic manipulators path planning. In: 2021 21st International Conference on Control, Automation and Systems. ICCAS, IEEE, pp. 512–517.
- Liu, B., Pu, Z., Pan, Y., Yi, J., Liang, Y., Zhang, D., 2023b. Lazy agents: a new perspective on solving sparse reward problem in multi-agent reinforcement learning. In: *International Conference on Machine Learning*. PMLR, pp. 21937–21950.
- Lopez-Franco, C., Hernandez-Barragan, J., Alanis, A.Y., Arana-Daniel, N., 2018. A soft computing approach for inverse kinematics of robot manipulators. *Eng. Appl. Artif. Intell.* 74, 104–120.
- Man, H., Ge, N., Xu, L., 2021. Intelligent motion control method based on directional drive for 3-DOF robotic arm. In: 2021 5th International Conference on Robotics and Automation Sciences. ICRAS, IEEE, pp. 144–149.
- Matiisen, T., Oliver, A., Cohen, T., Schulman, J., 2019. Teacher-student curriculum learning. *IEEE Trans. Neural Netw. Learn. Syst.* 31 (9), 3732–3740.
- Ni, X., He, X., Matsumaru, T., 2021. Training a robotic arm movement with deep reinforcement learning. In: 2021 IEEE International Conference on Robotics and Biomimetics. ROBIO, IEEE, pp. 595–600.
- Or, K., Wu, K., Nakano, K., Ikeda, M., Ando, M., Kuniyoshi, Y., Niiyama, R., 2023. Curriculum-reinforcement learning on simulation platform of tendon-driven high-degree of freedom underactuated manipulator. *Front. Robot. AI* 10, 1066518.
- Rivera-Borrorro, O.M., Rabassa-Gutiérrez, M., Grau-Ábalo, R.d.C., Marrero-Ponce, Y., García-de la Vega, J.M., 2012. Dunn's index for cluster tendency assessment of pharmacological data sets. *Can. J. Physiol. Pharmacol.* 90 (4), 425–433.
- Saglietti, L., Mannelli, S., Saxe, A., 2022. An analytical theory of curriculum learning in teacher-student networks. *Adv. Neural Inf. Process. Syst.* 35, 21113–21127.
- Shahapure, K.R., Nicholas, C., 2020. Cluster quality analysis using Silhouette score. In: 2020 IEEE 7th International Conference on Data Science and Advanced Analytics. DSAA, IEEE, pp. 747–748.
- Singh, A.K., Mittal, S., Malhotra, P., Srivastava, Y.V., 2020. Clustering evaluation by davies-bouldin index (dbi) in cereal data using k-means. In: 2020 Fourth International Conference on Computing Methodologies and Communication. ICCMC, IEEE, pp. 306–310.
- Soviany, P., Ionescu, R.T., Rota, P., Sebe, N., 2022. Curriculum learning: A survey. *Int. J. Comput. Vis.* 130 (6), 1526–1565.
- Tang, W., Cheng, C., Ai, H., Chen, L., 2022. Dual-arm robot trajectory planning based on deep reinforcement learning under complex environment. *Micromachines* 13 (4), 564.
- Tang, G., Hauser, K., 2017. A data-driven indirect method for nonlinear optimal control. In: 2017 IEEE. In: *RSJ International Conference on Intelligent Robots and Systems*. IROS, pp. 4854–4861.
- Wang, Y., Xu, Y., Gao, T., 2021. Evaluation method of wind turbine group classification based on Calinski Harabasz. In: 2021 IEEE 5th Conference on Energy Internet and Energy System Integration. EI2, IEEE, pp. 2630–2635.

- Waskom, M.L., 2021. Seaborn: statistical data visualization. *J. Open Source Softw.* 6 (60), 3021.
- Wen, S., Chen, J., Wang, S., Zhang, H., Hu, X., 2018. Path planning of humanoid arm based on deep deterministic policy gradient. In: 2018 IEEE International Conference on Robotics and Biomimetics. ROBIO, IEEE, pp. 1755–1760.
- Yang, A., Chen, Y., Naeem, W., Fei, M., Chen, L., 2021. Humanoid motion planning of robotic arm based on human arm action feature and reinforcement learning. *Mechatronics* 78, 102630.
- Yang, S., Wang, Q., 2022a. Robotic arm motion planning with autonomous obstacle avoidance based on deep reinforcement learning. In: 2022 41st Chinese Control Conference. CCC, IEEE, pp. 3692–3697.
- Yang, S., Wang, Q., 2022b. Robotic arm motion planning with autonomous obstacle avoidance based on deep reinforcement learning. In: 2022 41st Chinese Control Conference. CCC, IEEE, pp. 3692–3697.
- Zhou, D., Jia, R., Yao, H., 2021a. Robotic arm motion planning based on curriculum reinforcement learning. In: 2021 6th International Conference on Control and Robotics Engineering. ICCRE, IEEE, pp. 44–49.
- Zhou, D., Jia, R., Yao, H., Xie, M., 2021b. Robotic arm motion planning based on residual reinforcement learning. In: 2021 13th International Conference on Computer and Automation Engineering. ICCAE, IEEE, pp. 89–94.